The Journal of Systems and Software 158 (2019) 110419

Contents lists available at ScienceDirect

The Journal of Systems and Software

journal homepage: www.elsevier.com/locate/jss

Automated code-based test selection for software product line regression testing

Pilsu Jung[a], Sungwon Kang[a], Jihyun Lee[b,*]

[a] School of Computing, KAIST, 291 Daehak-ro, Yuseong-gu, Daejeon, South Korea
[b] Department of Software Engineering, Chonbuk National University, 567 Baekje-daero, Deokjin-gu, Jeonju-si, South Korea

ARTICLE INFO

Article history:
Received 12 July 2019
Revised 6 September 2019
Accepted 9 September 2019
Available online 10 September 2019

Keywords:
Product lines testing
Regression test selection
Software maintenance
Software evolution

ABSTRACT

Regression testing for software product lines (SPLs) is challenging and can be expensive because it must ensure that all the products of a product family are correct whenever changes are made. SPL regression testing can be made efficient through a test case selection method that selects only the test cases relevant to the changes. Some approaches for SPL test case selection have been proposed but either they were not efficient by requiring intervention from human experts or they cannot be used if requirements specifications, architecture and/or traceabilities for test cases are not available or partially eroded. To address these limitations, we propose an automated method of source code-based regression test selection for SPLs. Our method reduces the repetition of the selection procedure and minimizes the in-depth analysis effort for source code and test cases based on the commonality and variability of a product family. Evaluation results of our method using six product lines show that our method reduces the overall time to perform regression testing by 14.8% ~ 49.1% on average compared to an approach of repetitively applying Ekstazi, which is the state-of-the-art regression test selection method for a single product, to each product of a product family.

1. Introduction

Software testing is an important part of software development activity as it ensures the correctness and quality of a software product. Testing of a software product line (SPL) is more challenging than testing of a single product because it has to deal with a potentially large number of variants while trying to reduce redundancy of test executions (Lachmann et al., 2015).

SPL regression testing is performed to ensure that changes are correct and have not adversely affected the unchanged parts in the context of the SPL (Harrold and Orso, 2008). To perform SPL regression testing, we can rerun all the existing test cases that the target system has passed before the change is made. However, such an approach unnecessarily incurs high cost because many test cases would be rerun whenever a change is made. A promising direction of regression testing is to select and rerun only the test cases that are relevant to the change. The problem of selecting a subset of test cases for testing of the changed parts of a system is called *the regression test selection (RTS) problem* (Yoo and Harman, 2012).

A typical RTS method for a single software system is conducted in three phases (Gligoric et al., 2015): the *Analysis* (*A*) *phase* selects test cases for a retest, the *Execution* (*A*) *phase* executes the selected test cases and the *Collection* (*C*) *phase* collects information (e.g., control/data flow of source code and traces of test cases) from the current version to enable analysis for the next revision. Regression test cases for SPL can be selected by applying the *AEC* phases to each product of a product family. However, such an approach incurs cost that is proportionally to the number of products of the family and can be avoided by taking software product line approach that exploits their commonality and variability. To reduce this inefficiency, unnecessary repetitions of the *AEC* phases and in-depth analysis of source code should be avoided as much as possible.

In the context of an SPL, there has been little research on RTS, as pointed out in Lee et al. (2012), Neto et al. (2011) and exposed in the more recent survey papers listed in a tertiary study (Marimuthu and Chandrasekaran, 2017), and no research on RTS for an SPL at the source code level exists at all, in spite of its importance, as exposed through those papers (Lee et al., 2012; Marimuthu and Chandrasekaran, 2017; Neto et al., 2011). Some

* Corresponding author.
E-mail addresses: psjung@kaist.ac.kr (P. Jung), sungwon.kang@kaist.ac.kr (S. Kang), jihyun30@jbnu.ac.kr (J. Lee).

https://doi.org/10.1016/j.jss.2019.110419
0164-1212/

existing RTS methods for SPL (Lity et al., 2018; Lochau et al., 2012; Neto et al., 2010) can be applied at the source code level with certain adjustments but would not result in efficient solutions since they require the intervention by human experts or well managed SPL artifacts (e.g., requirements specification, architecture and/or traceabilities for test cases). Thus to make SPL regression testing efficient even when human experts or thoroughly managed SPL artifacts are not available, we need an automated RTS method for SPLs that does not require thoroughly managed artifacts.

This paper proposes an automated code-based RTS method for SPLs. It is the first one that selects regression test cases at the code level when changes are made to a product line code base, which is a set of code fragments to be used to produce products of a product line. The key insight in our method is to avoid unnecessary repetitions in the RTS procedure for the common code part of a product family and reduce the in-depth analysis effort of source code and test cases in the test selection process for the variable code part of the family. For the changes to the common parts of the code, our method selects test cases relevant to the changes by applying an RTS method for a single software system to a product family. Then it instruments the bytecode that prints dependencies of test cases (cf., Gligoric et al., 2015) to common code and executes the selected test cases on only one of the changed products. For the changes to the variable parts of the code, our method identifies the test cases that can never be affected by the changes and filters them out. With this process, our method reduces the overall cost of regression testing without missing any fault-revealing test cases when the Rothermel's assumptions (Rothermel and Harrold, 1996) hold and product line test runs are deterministic.

We conducted an experimental evaluation of our method using six product line systems. The results show that without missing any fault-revealing test cases (Rothermel and Harrold, 1996), our method reduces, on average, 14.8% $\sim$ 49.1% of the end-to-end time to perform regression testing by the Ekstazi_SPL, which is an application of Ekstazi (Gligoric et al., 2015) to SPL (cf. Section 7.1.2).

The rest of the paper is organized as follows. Section 2 describes the background knowledge necessary to present our method. Section 3 presents an overview of our method that selects regression tests. Sections 4 and 5 describe how our method reduces test cases for regression testing of the common part and the variable part of an SPL, respectively. Section 6 presents an implementation of our method. In Section 7, we evaluate our method. In Section 8, we discuss related work on regression testing. In Section 9, we conclude the paper.

2. Background

In this section, we present definitions, assumptions and basic concepts necessary for the presentation of our method in this paper.

2.1. Software product lines

2.1.1. Definition (product family)

A set of products that have a significant amount of commonality is called a *product family*.

2.1.2. Definition (platform artifact and variation point)

A platform artifact of a product family is an artifact that consists of the common parts of the product family and the abstractions of their variable parts, called *variation points*.

2.1.3. Definition (platform)

A platform of a product family is the set of platform artifacts for the product family.

The terms "platform" and "platform artifact" are widely used in the literature including text books such as Pohl et al. (2005) and van der Linden et al. (2007), survey papers such as Baker et al. (2015), Engström and Runeson (2011), Lee et al. (2012) and numerous research papers. The definition of "platform" above is more precisely formulated than its definitions in the existing literature such as that of Pohl et al. (2005), so that it describes clearly the essential substances of platform (i.e., the common parts of the product family and variation points).

2.1.4. Definition (binding information)

Binding information for a product of a product line is a set of selections, for the product, of specific variants for the variation points in the platform of the product line.

Binding information is defined to specify the artifacts (e.g., source code, test case, etc.) of planned products that are to be actually produced. Each binding contained in binding information is a set of substitutions of the form $vpj\backslash vj_i$ where vpj is a variation point, vj_i is one of variants of vpj, vj_1, ..., vj_n and '\' represents a substitution. A set of substitutions is denoted [$vp1\backslash v1_i$, ..., $vpm\backslash vm_i$] where vp1, ..., vpm are variation points and vj_i is the variant of vpj, $1 \leq j \leq m$, selected for a product. Once binding information is defined, a binding can be applied by performing set of substitutions to platform artifacts (e.g., source code, test case, etc.).

2.1.5. Definition (product line)

A product line is a product family that has a platform and binding information for its planned products.

2.1.6. Definition (product line code base)

Product line code base is the entire set of code fragments that is used to produce products of a product line.

In a product line, changes can be made not only to the product line code base, but also to the source code of products produced from the product line code base (Yu and Ramaswamy, 2006). In this paper, we focus on the changes to the product line code base because once such changes can be handled, the other type of changes can be handled in a similar way (Yu and Ramaswamy, 2006). Thus, our method is used for regression testing of a product line when its code base is changed.

2.1.7. Definition (generative product line)

A product line that defines bindings of product artifacts for all the planned products and can produce them by instantiating its platform with their respective bindings is called a *generative product line*.

Thus, if the platform of a product line cannot generate all the planned products by instantiating the platform, then it is a non-generative product line. Unlike a generative product line, a non-generative product line needs additional artifacts other than the platform in order to develop all the planned products.

The generative product line is widely used in practice. For example, the AEGIS product line of Lockheed Martin and the vehicle product line of General Motors are generative product lines (Gregg et al., 2017; Young et al., 2017). Well-known product line development tools such as FeatureIDE,[1] CIDE[2] and Pure::variants[3] support development of generative product lines. The research in this paper targeted generative product lines.

In product line development, products are developed by instantiating platform artifacts of a product line. Therefore, a product line defines binding information for each product that will allow product artifacts to be derived from platform artifacts

[1] http://www.featureide.com/.
[2] http://ckaestne.github.io/CIDE/.
[3] https://www.pure-systems.com/.

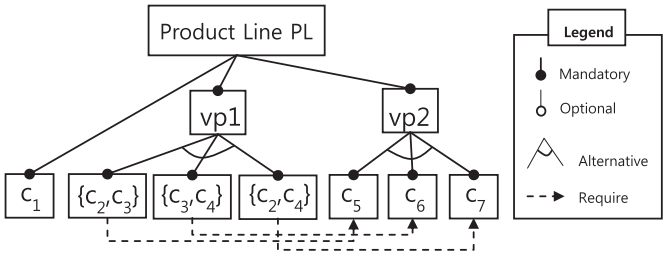


Fig. 1. A variability model of a product line system.

(Pohl et al., 2005). For example, Common Variability Language (CVL) (Haugen, 2012), which is a domain-independent language for specifying variability of a product line, requires binding information called a *resolution model*. This information is used to produce product artifacts called a product model from platform artifacts called a *product line model*. Also well-known existing product line development tools use binding information, typically referred to as configuration information, to produce products of a product family. Their binding information enables source code and test cases (artifacts) for each product to be produced from product line code base and test cases. For example, the FeatureIDE, Pure::variants and CIDE use, respectively, a configuration file (.config), configuration matrix and coloring file (.colors) to produce source code and test cases for each product of a product family. In practice, Lockheed Martin, General Motors and BigLever Software produce product artifacts for each planned product by exercising variation points according to binding information called the bill-of-features (Gregg et al., 2017; Young et al., 2017).

Following these practices of product line development, we focus on generative product lines in which, by application of the binding information for a product, its product artifacts, especially source code and test cases, are generated.

2.1.8. Definition (variant code fragment)

A variant code fragment is a piece of code that constitutes a part or the whole of a variant that is designed to be bound to a variation point of a product line artifact.

Thus a variant of a variation point is implemented by a set of variant code fragments. When a variant is bound to a variation point with binding information for a product, this variation point can be instantiated with the constituent variant code fragments for the product.

Fig. 1 shows a variability model in the notation, which extends that of Batory (2005), for the code base of a product line. To describe the code-level variability precisely and concisely, we allowed a set of variant code fragments to be contained in a single box. This model has two variation points vp1, vp2 and seven variant code fragments c_1, ..., c_7. In the variability model, c_1 is a mandatory variant code fragments; {c_2, c_3}, {c_3, c_4} and {c_2, c_4} are *alternative* sets of variant code fragments and are abstracted by vp1; c_5, c_6 and c_7 are *alternative* variant code fragments and are abstracted by vp2. The dotted line denotes the constraints between features. For example, the binding of {c_2, c_3} to vp1 requires the binding of c_5 to vp2.

Once a variability model is constructed, binding information based on it can be defined, applying the constraints between features. For example, the left hand side of Fig. 2 shows binding information B_{P1} B_{P2}, B_{P3} for planned products P1, P2 and P3 of the product line considered in Fig. 1. Then each of these products can be produced by application of its binding information, that is, by substituting vp1 and vp2 with the variant code fragments specified in its binding information. For example, P1 contains c_2, c_3 and c_5 in addition to c_1 since, by applying B_{P1}, vp1 is substituted by {c_2, c_3} and vp2 is substituted by c_5.

$$B_{P1}=[vp1\backslash\{c_2, c_3\}, vp2\backslash c_5] \Rightarrow P1=\{c_1,c_2,c_3,c_5\}$$
$$B_{P2}=[vp1\backslash\{c_3, c_4\}, vp2\backslash c_6] \Rightarrow P2=\{c_1,c_3,c_4,c_6\}$$
$$B_{P3}=[vp1\backslash\{c_2, c_4\}, vp2\backslash c_7] \Rightarrow P3=\{c_1,c_2,c_4,c_7\}$$

Fig. 2. An application of binding information.

To exemplify how variability is defined, how bindings are made and how source code and test cases are shared amongst products at the code level, we use an example implemented by Java language. To simplify the example, we use a small product line source class (i.e., Calculator class) and a few product line test cases (i.e., TestCalculator class).

To develop the Calculator SPL and its product family, a variability model, SPL artifacts (e.g., source code and test cases) and the traceability links between them are first developed. Fig. 3(a) represents the variability model and Figs. 3(b) and 3(c) represent the SPL artifacts. To develop a traceability link at the code level, FeatureIDE creates a unique directory for variant code fragments, Pure::variants links variant code fragments with a unique file and CIDE assigns unique colors to variant code fragments. However, to simplify our presentation, we just tagged the identifiers of the variant code fragments next to the line numbers. Then, binding information is defined to specify a set of planned products to be produced. Fig. 3(d) is the binding information for the planned products, P1, P2 and P3. Finally, the planned products are produced by applying their binding information to the variability model. In the example, the variant code fragments c_1, ..., c_7 are deployed and the three planned products, P1, P2 and P3, are produced as shown in Fig. 3(e). In the source code of the product family, the *sum* and *sub* methods are common to all the products; the *mult* method is common to P1 and P3; the *div* method is common to P1 and P2; the *mod* method is common to P2 and P3; the *testCalc1*, the *testCalc2* and the *testCalc3* methods are specific to P1, P2 and P3, respectively.

2.2. Regression testing

So far we presented the fundamental concepts related to product line development. In the rest of this section, we present definitions and assumptions related to product line regression testing.

2.2.1. Definition (fault-revealing test case (Rothermel and Harrold, 1996))

A test case for a program that causes its modified program to fail is called a *fault-revealing test case* for the modified program.

If *P* is a program, *P'* is its modified program and *T* is a set of test cases to test *P*, then fault-revealing test cases for *P'* can be identified by finding the test cases that traverse the changes made to *P* (to be called *modifications for P'*) under the following assumptions introduced in Rothermel and Harrold (1996):

2.2.2. P-*Correct-for-*T *assumption*

When *P* is tested with $t \in T$, *P* halts and produces the correct output.

2.2.3. Obsolete-Test-Identification assumption

There exists an effective procedure for determining whether $t \in T$ is obsolete for *P'*.

2.2.4. Controlled regression testing assumption

When *P'* is tested with *t*, all factors that might influence the output of *P'* except for the code in *P'* are constant with respect to their states when *P* was tested with *t*.

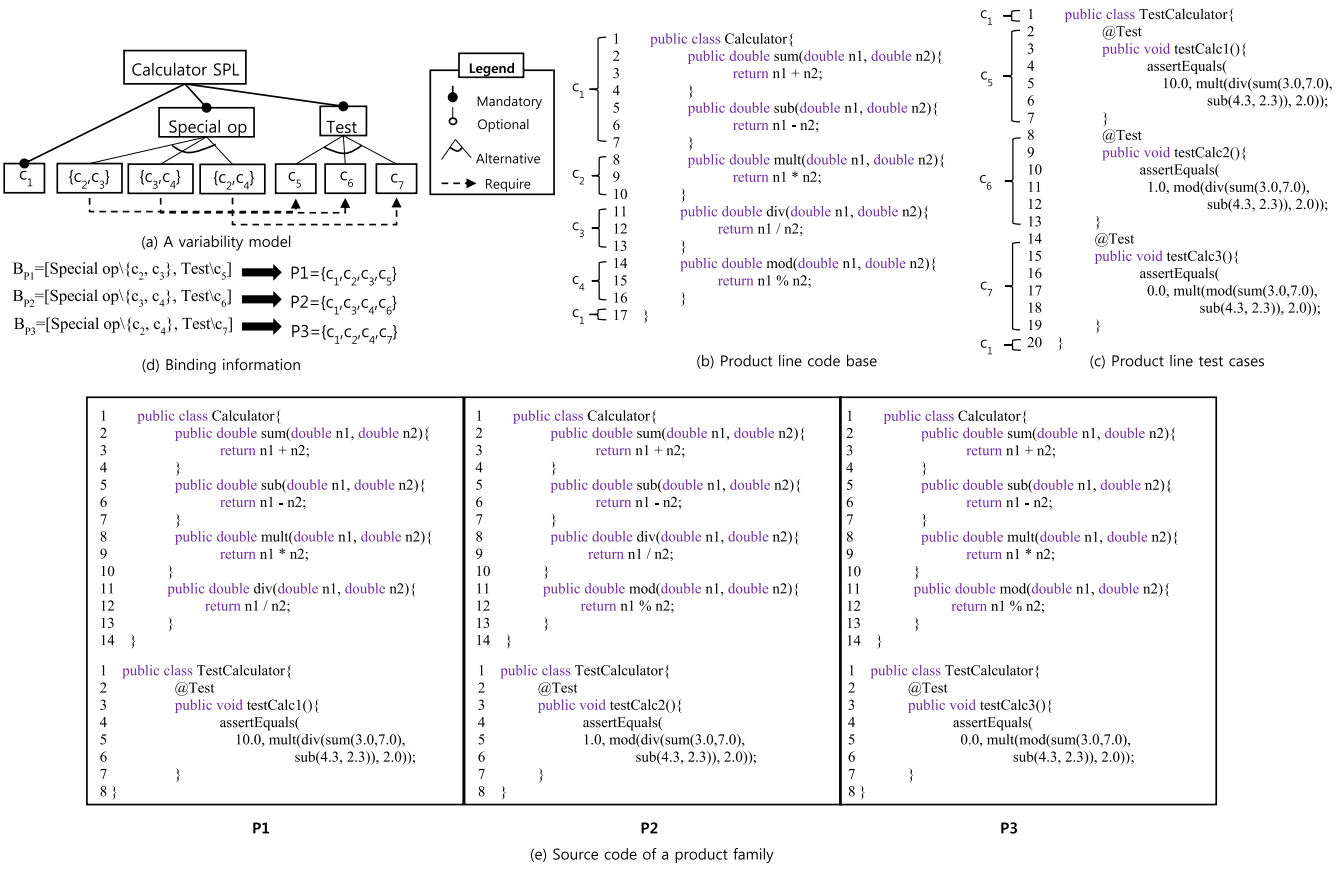


Fig. 3. Example of developing the Calculator SPL at the code level.

2.2.5. Definition (change-relevant test case)

A change-relevant test case is a test case that tests the changed parts of the source code.

When a change is made, change-relevant test cases should be rerun because they include all of the fault-revealing test cases, which can reveal faults caused by the change (Rothermel and Harrold, 1996).

2.2.6. Definition (change-irrelevant test case)

When a change is made, a change-irrelevant test case denotes a test case that tests only the parts of source code not affected by the change.

Change-irrelevant test cases are not useful for finding faults caused by changes because they are not fault-revealing test cases. Therefore, such test cases should be left out in order to efficiently retest the changed system.

Typically, a product line test case is reused for the products of a product family. In this case, it must be guaranteed that a test case traverses the same set of statements and produces the same output on its reused products. Otherwise, its testing result is not reusable for the products because the test case can behave differently over different products. Therefore, we assume a deterministic test runs as follows:

2.2.7. Deterministic product line test run assumption

A product line test run is deterministic if it covers the same set of statements and produces the same output 1) whenever it is run on the same product and 2) when it is run on its applicable products of a product family.

In this paper, our method assumes that a product line test run is deterministic. A test case can have non-deterministic runs by various reasons, such as multi-threading, asynchronous calls, networking, concurrency, etc. In addition, a product line test case may be designed such that it tests different code parts depending on the product under test. For example, if one of variability of a product family is a sort algorithm and its product line test case tests Merge Sort function in product A and Quick Sort function in product B, the test case covers a different set of statements depending on which product is tested even though products of a product family individually behave deterministically. However, by the deterministic product line test run assumption, a test case covers the same set of statements and always produces the same results in different runs and in different products if a product is unchanged. Therefore, in the above example, the two different sort functions are tested by each of two different test cases. This assumption limits the application scope of our method to be developed in this paper but we will relax this assumption in the future work to deal with non-deterministic test runs.

3. The automated code-based test case selection method for product line regression testing

In this section, we present an overview of our method for product line RTS together with additional definitions to present our method.

3.1. Definitions

In product line development, a product line should have its code base partitioned into fragments such that certain of their combinations (or integrations) constitute products (Pohl et al., 2005). Furthermore, typically, since the planned products produced

from a product line should be independently and individually executed, each of the pieces should not have any dependency relationship with other pieces except for those of a product containing it. Furthermore, for development cost reduction, it is important to have as much commonality as possible and to reduce the amount of variability (Pohl et al., 2005). Following this practice, we define the notion of a well-developed product line code base. First, we say (one part of) source code S1 *uses* (another part of) source code S2 if S1 contains a call of a function in S2 or an access of a variable in S2.

3.1.1. Well-developed product line code base assumption

Product lines considered in this paper are well-developed product lines where the source code for two or more products is developed only once and reused for those products (Condition 1), and the source code uses only the source code of a product that contains it (i.e., if P_1, ..., P_k are all the products to which a code fragment c belongs to, then c can use only c_1, ..., c_n where $\{c_1,...,c_n\}$ is the set of code fragments of P_j, $1 \le j \le k$) (Condition 2).

For example, by Condition 1 of this definition, if a set of variant code fragments, c_1, ..., c_7, in Fig. 1 contains two or more equivalent code fragments, we say that these code fragments are duplicated and the product line code base is not well developed. A product line code base that violates Condition 1 cannot reduce the development cost much and becomes vulnerable to change due to its duplicated source code. By Condition 2, if the product line code base is developed such that a method m_1 of P1 uses a method m_2 that is not in P1, we say that it is not well developed. For a product line code base to be well developed, it should be developed such that m_1 uses only the methods of P1. A product line code base that violates Condition 2 produces invalid products that are not executable. For these reasons, we assume in this paper that the product line code base we consider is well developed. Assuming a well-developed product line is reasonable because, in the context of SPL, exploiting commonality to the maximal extent is desirable for cost reduction (Pohl et al., 2005) and invalid uses among source code should be avoided. Moreover, there have been extensive studies on code clone detection (Rattan et al., 2013) and product line refactoring (Laguna and Crespo, 2013), which can be applied to refactor a product line that is not well developed into a well-developed product line.

Now we define the notions of maximality of a set of variant code fragments and space, which play crucial roles in our method.

3.1.2. Definition (maximality of a set of variant code fragments)

For a product line code base $C=\{c_1, c_2, ..., c_m\}$ where c_1, c_2, ..., c_m are variant code fragments, let P_1, P_2, ..., P_n be a family of products that are produced from C. And let $S_1 \subseteq C$ be a set of variant code fragments that are reused for some products P_1, ..., P_k of the family. Then we say that S_1 is common to P_1, ..., P_k. If $S_1 \subseteq C$ is common to P_1, ..., P_k and, for any $c \in C$ such that $c \notin S_1$, $S_1 \cup \{c\}$ is not common to P_1, ..., P_k, then we say that S_1 is maximally common to P_1, ..., P_k. Let $S_2 \subseteq C$ be a set of variant code fragments that are used for and only for a single product P_x of the family. Then we say that S_2 is unique to P_x. If $S_2 \subseteq C$ is unique to P_x and, for any $c \in C$ such that $c \notin S_2$, $S_2 \cup \{c\}$ is not unique to P_x, then we say that S_2 is maximally unique to P_x.

For example, let us suppose that the code base of a product line consists of variant code fragments c_1, ..., c_8 and from this product line code base, planned products P1, P2, and P3 are produced where $P1=\{c_1, c_2, c_3, c_4, c_5, c_7, c_8\}$, $P2=\{c_1, c_2, c_3, c_6\}$, and $P3=\{c_1, c_2, c_4, c_5, c_6\}$, based on some source code binding information. In this case, $\{c_1, c_2\}$ is maximally common to P1, P2, and P3 because it is reused for and only for P1, P2, and P3 and no other variant code fragments are reused for and only for them. $\{c_4, c_5\}$ is maximally common to P1 and P3 because they are reused for and only for P1 and P3 and no other variant code fragments are reused for and only for them. $\{c_7, c_8\}$ is maximally unique to P1 because they are used for P1 and not for P2 and P3, and no variant code fragments other than c_7 and c_8 are used for and only for P1. But $\{c_1, c_2\}$ is not maximally unique to P1 because $\{c_1, c_2\}$ is used for P1 but also for P2 and P3.

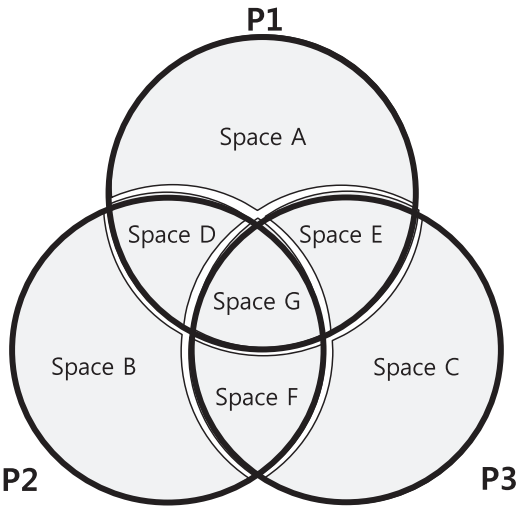


Fig. 4. Product line code base partitioned into spaces according to the commonality and variability of products.

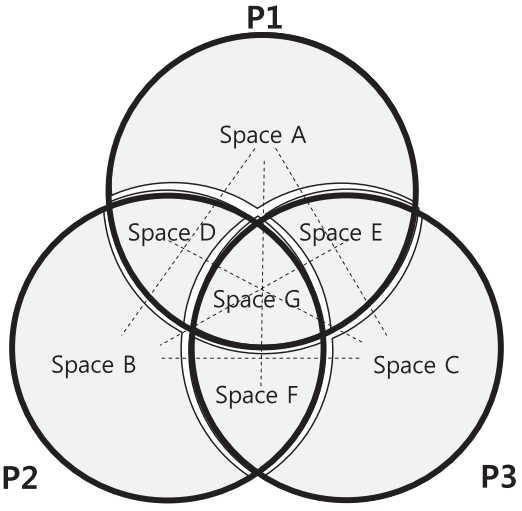


Fig. 5. Use relationships among spaces that should be excluded in well-developed product line.

3.1.3. Definition (space)

Let *X* be the set of all the variant code fragments of a well-developed product line. If *X* is partitioned into P such that each $s \in P$ is maximally common to a set of products or maximally unique to a single product, then *s* is a *space* of the product line code base.

Fig. 4 shows a graphical example of the spaces partitioned from the product line code base that produces products P1, P2 and P3. P1 consists of Spaces A, D, E and G; P2 consists of Spaces B, D, F and G; and P3 consists of Spaces C, E, F and G. Of these spaces, A, B and C consist of a code unique to a single product, and D, E, F and G consist of a code common to two or more products. The spaces A, B, C, D, E, F and G do not have overlapping with each other. By the assumption of a well-developed product line code base, there are use relationships not allowed among spaces because a space can use only the spaces of the product that contains the space. Fig. 5 shows such use relationships with dotted lines. A product

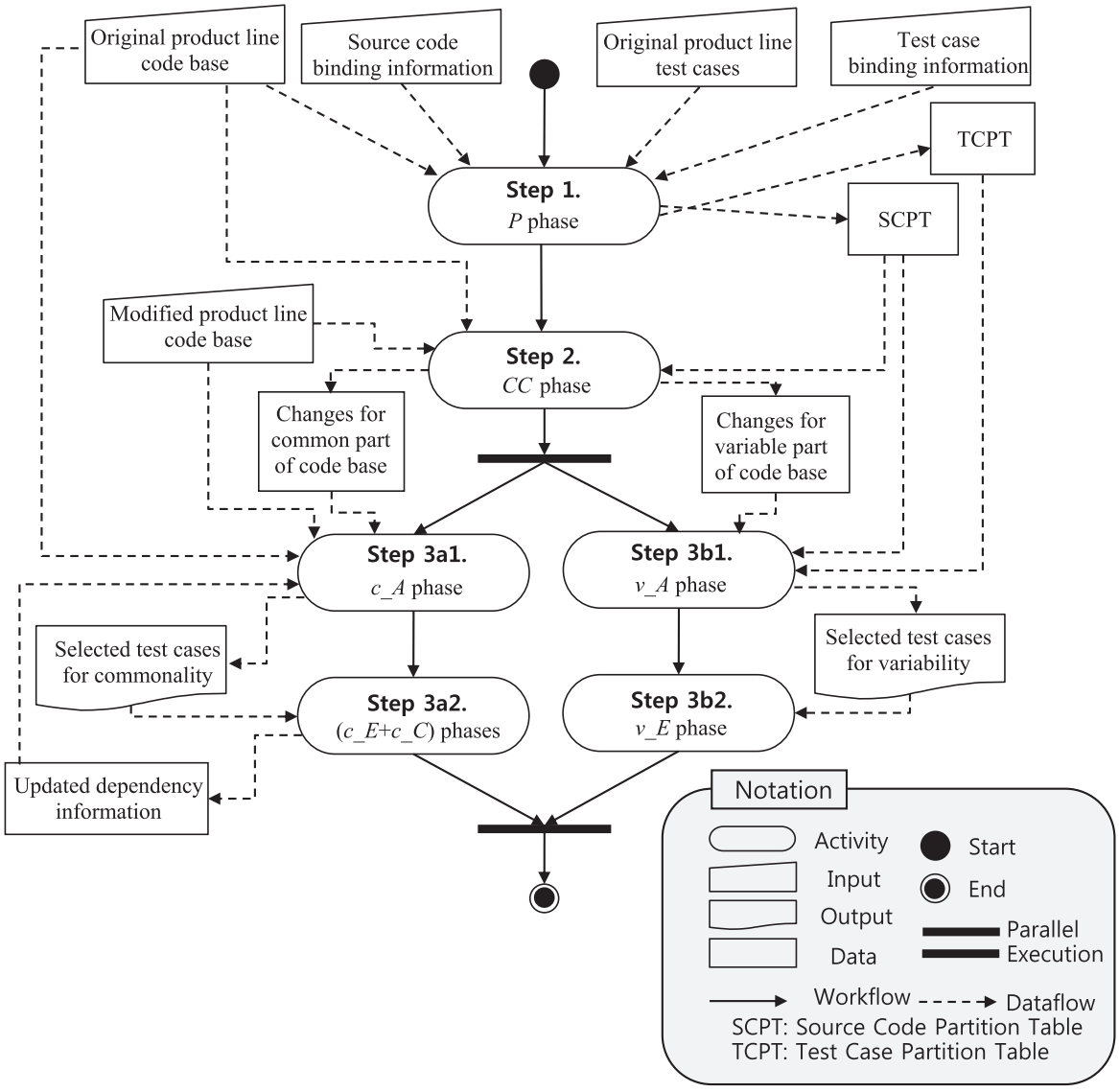


Fig. 6. Overview of our RTS method for SPL.

line containing such use relationships produces products with invalid uses. For example, if in a product line code base, Space A uses Spaces B, C or F, then it produces invalid products because B, C and F are not spaces of P1. However, A can safely use A, D, E or G.

Our method uses binding information for planned products to partition a product line code base into spaces. Such partitioning is always possible as can be seen in the following example. In Figs. 1 and 2, binding information B_{P1}, B_{P2} and B_{P3} represent P1 = {c_1, c_2, c_3, c_5}, P2 = {c_1, c_3, c_4, c_6}, and P3 = {c_1, c_2, c_4, c_7}. We notice that variant code fragments for the product line with three products can be grouped as follows: one that consists of variant code fragments common to all three products, one that consists of the sets of variant code fragments common to exactly two products; and one that consists of the sets of variant code fragments unique to a single product. Therefore, for the set {c_1, c_2, c_3, c_4, c_5, c_6, c_7} in Figs. 1 and 2, they are, respectively, {c_1}, {{c_2}, {c_3}, {c_4}} and {{c_5}, {c_6}, {c_7}}, and since each of {c_1}, ..., {c_7} is maximally common to a set of products or maximally unique to a single product, each of them is a space. Thus, c_1 makes a space common to P1, P2 and P3; {c_2}, {c_3} and {c_4} make spaces common to, respectively, both P1 and P3, both P1 and P2, and both P2 and P3; {c_5}, {c_6} and {c_7} make spaces unique to P1, P2 and P3, respectively.

For subsequent discussion of this paper, we call a source code common to all the planned products the *common part of the source code* and the other source code the *variable part of the source code*. For example, in Fig. 4, Space G is a common part of the source code and the other spaces are the variable parts of the source code.

3.2. Overview of our method

As the top part of Fig. 6 shows, the inputs of our method are as follows: (1) the original product line code base, (2) the modified product line code base, (3) the original product line test cases and (4) the binding information for each product. The output of our method is the test cases selected for a retest. The binding information for the test cases of a product includes binding information for the source code of the product but may contain additional binding information for test specific aspects such as the test method, test coverage and test architecture. Therefore, we consider both kinds of binding information separately as shown at the top of Fig. 6.

Step 1 partitions product line code base and test cases based on commonality and variability of a product family. Step 2 determines whether a change has been made to the common part of the source code or the variable part of the source code. Finally, Step 3 selects test cases for a retest. Step 3 is divided into two sequences

Table 1
SCPT for the family in Fig. 4.

Space	Set of products (key)	Variant code fragments (value)
Space A	{P1}	Variant code fragments for P1 only
Space B	{P2}	Variant code fragments for P2 only
Space C	{P3}	Variant code fragments for P3 only
Space D	{P1,P2}	Variant code fragments for P1 and P2 but not P3
Space E	{P1,P3}	Variant code fragments for P1 and P3 but not P2
Space F	{P2,P3}	Variant code fragments for P2 and P3 but not P1
Space G	{P1,P2,P3}	Variant code fragments for all products

Table 2
TCPT for the family in Fig. 4.

Test suite	Set of products (key)	Test cases (value)
TS1	{P1}	Test cases applicable to P1 only
TS2	{P2}	Test cases applicable to P2 only
TS3	{P3}	Test cases applicable to P3 only
TS4	{P1,P2}	Test cases applicable to P1 and P2 but not P3
TS5	{P1,P3}	Test cases applicable to P1 and P3 but not P2
TS6	{P2,P3}	Test cases applicable to P2 and P3 but not P1
TS7	{P1,P2,P3}	Test cases applicable to all products

of substeps that can be executed in parallel, i.e., the sequence of Step 3a1 and Step 3a2 and the sequence of Step 3b1 and Step 3b2 as shown in the bottom part of Fig. 6. Step 3a1 and Step 3a2 handle the changes made to the common part of source code and Step 3b1 and Step 3b2 handle the changes made to the variable part of source code.

A set of test cases selected from the two sequences of our method always includes all the fault-revealing test cases because each sequence selects the test cases that traverse changes and, as stated in Section 2, the fault-revealing test cases can be selected by finding the test cases that traverse changes. Therefore, our method always selects all the fault-revealing test cases even though it individually deals with changes for the common part of the source code and changes for the variable part of the source code.

In this section, we present Steps 1 and 2. The rest of the steps are presented in Sections 4 and 5.

3.3. Step 1. Partitioning phase (P phase)

Step 1 partitions a product line code base into spaces. Also Step 1 partitions all product line test cases into disjoint sets of product line test cases such that each set of product line test cases is applicable to a (nonempty) subset of all planned products (and only to the products in the subset) and to each subset of all planned products (and only to the products in this subset) there is one set of product line test cases that is applicable. To realize this step, the *source code partition table* (SCPT) and the *test cases partition table* (TCPT) are used. As Tables 1 and 2 show, SCPT and TCPT contains pairs of key and value for spaces and for test suites, respectively, such that they have one pair for one space or one test suite. Nonempty subsets of all planned products become the keys for the pairs of both SCPT and TCPT. The value part of a pair in SCTP contains a set of variant code fragments that is maximally common or maximally unique to the subset. If a set of variant code fragments is maximally common to two or more subsets of all planned products, then SCTP contains only the pair whose key part has the largest such subset. The value part of TCPT contains the set of test cases applicable to and only to the subset.

The **buildSourceCodePartitionTable** function in Fig. 7 shows our source code partitioning algorithm. The algorithm first extracts all variant code fragments (CF_{SPL}) from the original product line code base (C_{SPL}) based on the source code binding information ($B_{C_{SPL}}$) (line 2). Next, the algorithm selects the largest set of planned products (S) containing each variant code fragment (cf) based on the source code binding information ($B_{C_{SPL}}$) (lines 4–5). S is used as the identifier of a space and also a key of the SCPT. Finally, the algorithm puts cf in the value part of S ($SCPT[S]$) (line 6). When the algorithm is finished, each value part of SCPT forms a space because each set of variant code fragments in the value part of SCPT is maximally common to a product set or maximally unique to a single product. In the same way, the TCPT can be built based on the **buildTestCase-PartitionTable** function of Fig. 7. Tables 1 and 2 respectively present the SCPT and the TCPT of the product family {P1, P2, P3} as defined in Fig. 4.

3.4. Step 2. Change classification phase (CC phase)

Step 2 identifies changes from the two versions of product line code base and classifies the changes into changes made to the common part of the source code and changes made to the variable part of the source code using the SCPT built in Step 1. The changes are made in the variant code fragments (CF_{SPL}) extracted in Step 1. For example, Fig. 8 presents the product line code base implemented on CIDE. The colored one represents the variable part of source code and the uncolored one represents common part of source code. In the *C* class, the qualifier of the *result* field has been changed from *private* to *public* (line 2) and the body of the *foo* method has also been partially changed (line 5). In this case, because the variant code fragments in this example is the method/field level, CF_{SPL} contains the *foo* method, the *bar* method and the *result* field; $CF_{changed}$ contains the *foo* method and the *result* field. After the changes are identified, they are classified into two types of changes: a set of changes made to the common part of the source code (i.e., *result* field) and a set of changes made to the variable part of the source code (i.e., *foo* method). The two sets of changes are used as an input for Step 3a1 and Step 3b1, respectively.

4. Test case selection based on the commonality of a product family

A typical RTS method for a single software system has three phases (Gligoric et al., 2015): the *Analysis* (*A*) *phase* identifies differences between original and changed versions and selects tests to rerun, the *Execution* (*E*) *phase* runs the selected tests, and the *Collection* (*C*) *phase* collects information for the changed version.

Declaration: *SCPT:* map <key: set of products, value: set of variant code fragments>
Declaration: *TCPT:* map <key: set of products, value: set of test cases>
Input: C_{SPL}: product line code base
Input: T_{SPL}: product line test cases
Input: $B_{C_{SPL}}$: binding information of C_{SPL}
Input: $B_{T_{SPL}}$: binding information of T_{SPL}
Output: *SCPT*: source code partition table
Output: *TCPT*: test cases partition table
Use : traceCode(): input a code binding information and a variant code fragment and return a largest set of products which include the code fragment
Use : traceTC(): input a test case binding information and a test case and return a largest set of products which include the test case

1. **function** buildSourceCodePartitionTable(C_{SPL}, $B_{C_{SPL}}$)
2. Set CF_{SPL} ← extract all the variant code fragments from C_{SPL} based on in $B_{C_{SPL}}$
3. **foreach** $cf \in CF_{SPL}$ **do:**
4. Set $S \leftarrow \emptyset$
5. $S \leftarrow$ traceCode($B_{C_{SPL}}$,*cf*)
6. *SCPT[S]* puts *cf*
7. **endfor**
8. **return** *SCPT*
9. **end** buildSourceCodePartitionTable
10. **function** buildTestCasePartitionTable(T_{SPL}, $B_{T_{SPL}}$)
11. Set TC_{SPL} ← extract all the test cases from T_{SPL} based on $B_{T_{SPL}}$
12. **foreach** $t \in TC_{SPL}$ **do:**
13. Set $S \leftarrow \emptyset$
14. $S \leftarrow$ traceTC($B_{T_{SPL}}$.*t*)
15. *TCPT[S]* puts *t*
16. **endfor**
17. **return** *TCPT*
18. **end** buildTestCasePartitionTable

Fig. 7. Partitioning algorithm.

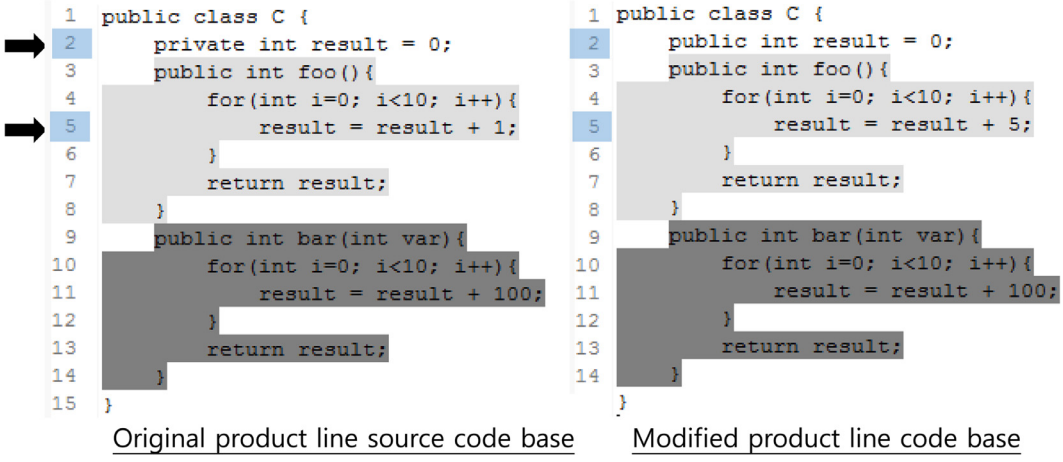


Fig. 8. An example of a change set.

For regression testing for SPL, these phases can be applied to each product of a product family. However, it incurs redundant work by applying the *AEC* phases multiple times to all the products of a product family. Our key idea in this part of our method is to reduce unnecessary repetitions by reusing the common results of the *AEC* phases for products of a product family when a change is made to the common part of the source code.

For changes made to the common part of the source code, our method applies an RTS method for a single software system to the SPL with some adjustments to reduce unnecessary cost. For that purpose, we adopt Ekstazi (Gligoric et al., 2015) because it is the state-of-the-art RTS method for a single software system and it is lightweight in that the overhead for test case selection is very small compared to the other existing RTS methods.

In the *A* phase, Ekstazi checks, for each test class, whether the checksums of all used source classes are the same. If so, no test class is selected. Otherwise, the test classes that have a dependency with the changed source classes are selected. Initially, on the first run, there is no dependency information for any test class. So every test class is selected. In the *E* phase, the test cases selected in the *A* phase are executed and rerun to check their testing results and update their dependency information. In the *C* phase, Ekstazi instruments the bytecode and then collects dependencies between test classes and source classes.

Ekstazi uses methods and classes as its selection granularity, which is the level where test cases are selected. However, prior studies (Gligoric et al., 2015; Vasic et al., 2017) have shown that class selection granularity reduces more testing costs compared to

Input: $TS[p_i]$: set of test cases for a product p_i
Input: P_{family}: product family of a product line
Input: T_{sel}: set of selected test cases
Output: set of selectively instrumented products
Use: InstCommonClasses(): input a set of classes of a product and instrument to only common source classes

1. **function** selectivelyInstrument(TS, P_{family}, T_{sel})
2. **foreach** $T_{sel} \neq \emptyset$
3. find p_i in P_{family} where $|TS[p_i] \cap T_{sel}|$ is max
4. $P_{family} \leftarrow P_{family} - p_i$
5. $T_{sel} \leftarrow T_{sel} - TS[p_i]$
6. InstCommonClasses(p_i)
7. **endfor**

Fig. 9. Selective instrumentation algorithm.

the method selection granularity. Therefore in this paper, we use the class selection granularity.

4.1. Step 3a1. Analysis phase (c_A phase)

Our method follows the analysis procedure of Ekstazi. However, unlike the Ekstazi, our method computes the checksums of source classes containing the common part of the source code because the classes that do not contain the common part of the source code consist of only variable parts of the source code, which will be handled in Steps 3b1 and 3b2.

4.2. Step 3a2. Execution phase (c_E phase)

In this step, the test cases selected in the *c_A* phase are executed. By the deterministic test run assumption, test classes that were not selected in the *c_A* phase need not be rerun because their testing results and dependencies will not be changed. On the other hand, the selected test classes should be rerun to check their testing results and to update their dependency information. Moreover, the testing result of a product can be reused for certain products of a product family because a test case always produces the same result on its applicable products. Therefore a test case run in a product need not be rerun on the other products. For this reason, our method executes the selected test cases on a single product.

4.3. Step 3a2. Collection phase (c_C phase)

Ekstazi collects dependencies between test classes and source classes. However, for an SPL, it is very redundant to apply the *C* phase multiple times to all the products of a product family. To reduce this redundancy, our method instruments to only common source classes of a product family because our method does not require dependency information for variable source classes (See Section 5). Moreover, our method instruments only to a subset of products where all the test cases selected in the *c_A* phase can be executed at least one time because by the deterministic test run assumption, dependency information of a test case is the same for its applicable products. Fig. 9 shows the algorithm of a selective instrumentation. For each product of a product family, the algorithm finds a product that has not been previously selected and contains the highest number of test cases selected in the *c_A* phase (lines 3–5). After that, it instruments the bytecode that collects dependency information to each common source class (line 6). For example, let us suppose that t1 is a test case of P1, t2 is a test case of P2 and t3 is a test case of both P1 and P3. In this case, if t1, t2 and t3 are selected in the *c_A* phase, only P1 and P2 are instrumented. P3 need not be instrumented because the dependencies of t3 can be obtained from P1.

The phase is performed together with the *c_E* phase in Step 3a2 because the dependency information is collected by executing test cases on the instrumented products.

5. Test case selection based on the variability of a product family

For changes made to the variable part of source code, our method selects test cases for a retest based on the variability of a product family. Our key idea is that the test cases just traversing the parts of the source code that are not affected by a change can be identified based on the variability of a product family and excluded from the selected test cases without in-depth analysis of the source code and test cases.

When a test case is executed, its execution path consists of function calls. We say the test case traverses the function calls (Rothermel and Harrold, 1996), and we also say the test case traverses the space that contains the code for the functions. We first present a lemma necessary to present our method.

Lemma 1. *If a product line code base is well-developed, a test case of the product line cannot traverse two different spaces unless they are spaces of the same product.*

Proof. By Condition 1 of a well-developed product line code base, the source code for two or more products is developed only once and reused across the products. Therefore, a test case is reused for testing of the source code reused, but not for testing of any other source code. Moreover, since a space can use only the spaces of the products that contain the space by Condition 2 (i.e., if P_1, ..., P_k are all the products to which a space s belongs to, then s can use only s_1, ..., s_n where $\{s_1,..., s_n\}$ is the set of spaces of P_j, $1 \leq j \leq k$), a test case cannot be designed to traverse two different spaces that are not spaces of the same product. Therefore, Lemma 1 holds. □

In Fig. 4, Spaces D and G are the spaces of product P1 (or P2) and a test case can traverse both Spaces D and G. However, a test case cannot traverse both Spaces A and B because they are not spaces of any one product.

5.1. Running example of test case selection based on space

Using Lemma 1, change-relevant test cases can be selected from a given set of test cases. There are three cases of change for product line code base.

Case 1. Change to source code unique to a single product

Let us suppose that the source code of Space A is changed. Then the test cases affected by the change should be rerun. The retest-all method would rerun all the test cases of P1, P2 and P3. However, it is costly to rerun all of them whenever a change occurs. Therefore, to efficiently retest the changed source code, we need to leave out the change-irrelevant test cases for Space A. By Lemma 1, the test cases traversing Spaces B, C or F cannot traverse Space A together. Such test cases are the change-irrelevant test cases because the test cases that do not traverse the changed part (i.e., Space A of this example) do not expose faults. Therefore, the test cases that traverse Spaces B, C or F need not be rerun.

By Lemma 1, we can select and rerun only the test cases traversing either Space A alone or both Space A and Spaces D, E and/or G together. Such test cases are applicable to P1 only since the test cases traversing Space A are not applicable on P2 and P3. As a result, when the source code in Space A is changed, it is sufficient to rerun only the test cases applicable to only P1.

Case 2. Change to source code common to two or more products but not to all the products

Let us suppose that the source code of Space F is changed. Then all the test cases affected by the changes must be rerun. To efficiently retest the changed source code, we leave out the change-irrelevant test cases for Space F. By Lemma 1, the test cases traversing Space A cannot traverse Space F. Therefore, the test cases that traverse Space A do not need to be rerun because they are the change-irrelevant test cases.

By Lemma 1, we can select and rerun only the test cases traversing either Space F alone or both Space F and Spaces B, C, D, E, F and/or G together. Such test cases are not applicable to P1 because the test cases traversing Space F are not applicable on P1. As a result, when the source code in Space F is changed, it is sufficient to rerun only the test cases applicable to P2 or P3 but not to P1.

Case 3. Change to source code common to all products

Let us suppose that the source code of Space G is changed. In this case, since every space can traverse Space G, we cannot identify the change-irrelevant test cases based on Space G and so cannot leave out any test cases. Therefore, test case selection based on space will be not adequate when change is made to the source code common to all the products of a product family.

5.2. Step 3b1. Test case selection phase (v_A phase)

This step identifies a set of changed spaces and then selects regression test suites from the TCPT built in Step 1. First, for changes identified in Step 2, their spaces are retrieved from the SCPT built in Step 1. For instance, if a change is made in the variant code fragments included in only P1 out of the products shown in Fig. 4, Space A (i.e., Space {P1}) is retrieved.

After a set of changed spaces is identified, a set of test cases for a retest is selected. In Section 2, we showed that the test cases applicable to the products that do not include the changed space do not need to be rerun because they are the change-irrelevant test cases by Lemma 1. These test cases can be straightforwardly identified through the TCPT. For example, in Fig. 4, when Space F is changed, we showed that the test cases applicable to P1 are change-irrelevant test cases because Space F belongs to P2 and P3, but not to P1 (shown in Case 2 of Section 5.1). These test cases can be retrieved from the TCPT by searching the test suites applicable to P1. The result is TS1, TS4, TS5 and TS7. We leave out these test suites. As a result, the regression test suites selected by our method are TS2, TS3 and TS6. This result can also be derived using the power set operator. For example, a power set of {P2, P3}, which is the set of products Space F belongs to, is {∅, {P2}, {P3}, {P2, P3}}. We can obtain the same result as test suites TS2, TS3 and TS6 by selecting the test suites corresponding to each element of the power set from the TCPT.

Our algorithm is shown in the **selectAffectedTestCases** function of Fig. 10. The algorithm first calculates the power set (*PS*) of the set of products (*S*) that contain each changed space (lines 6–10). Then it obtains the regression test suites (*R*) by retrieving the test suite corresponding to each element of the power set from the TCPT (lines 11–15).

5.3. Step 3b2. Test case execution phase (v_E phase)

In this step, the test cases selected in the *v_A* phase are executed. The test cases that are not selected in the *v_A* phase do not reveal any new fault because their testing results will be the same as the version before changes are made. Therefore, in this phase, the test cases selected by the *v_A* phase are executed. As in the *c_E* phase, by our deterministic test run assumption, a test case produces the same result on its applicable products. Therefore, we can execute the test case selected by the *v_A* phase on a single applicable product and reuse the result for the other products of the product family.

```
Declaration: SCPT: map <key: set of products, value: set of variant code fragments>
Declaration: TCPT: map <key: set of products, value: set of test cases>
Input: C_SPL: product line code base
Input: T_SPL: product line test cases
Input: B_C_SPL: binding information of C_SPL
Input: B_T_SPL: binding information of T_SPL
Input: CF_changed: set of changed variant code fragments
Output: R: set of affected test suite
Use: powerset(): input a set and return the power set of the set
1.  function selectAffectedTestCases(C_SPL, T_SPL, B_C_SPL, B_T_SPL, CF_changed)
2.    SCPT ← buildSourceCodePartitionTable(C_SPL, B_C_SPL)
3.    TCPT ← buildTestCasePartitionTable(T_SPL, B_T_SPL)
4.    Set Q ← ∅ // set of set
5.    Set S ← ∅
6.    foreach c ∈ CF_changed do:
7.          S ← key of SCPT whose value includes c
8.          PS ← powerset(S)
9.          Q ← Q ∪ PS
10.   endfor
11.   R ← ∅
12.   foreach q ∈ Q do:
13.         R ← R ∪ TCPT[q]        //lookup
14.   endfor
15.   return R
16. end selectAffectedTestCases
```

Fig. 10. Test cases selection algorithm.

6. Implementation

We implemented a prototype tool for our method named SRTST (Space based Regression Test Selection Tool for software product lines) and evaluated our method using it. Fig. 11 shows the architecture of SRTST, which consists of four components: *Partitioner*, *Change identifier*, *Collector*, and *Selector*.

The Partitioner component generates the SCPT and the TCPT from the product line code base and test cases based on binding information. SRTST uses a configuration file (.config) and a coloring file (.color) as binding information used respectively by FeatureIDE and CIDE. Through these files, SRTST obtains variant code fragments and test cases for each planned product. SRTST reads as test cases the methods annotated with '@Test'. Fig. 12 shows an example of a test case that SRTST reads in.

The Change identifier component first identifies a set of changes made to the product line code base. Next, this component classifies changes into changes made to the common part of the source code and changes made to the variable part of the source code using the SCPT. After that, for each change made to the common part of the source code, it identifies the source class that the change is made through a checksum comparison. For each change made to the variable part of the source code, it identifies the space that the change is made based on the SCPT.

The Collector component instruments bytecode, which prints dependency information to the common source classes of a product family. SRTST considers classes with the same name as common source classes. The bytecode is instrumented to the constructor, static method and static field of every class. After that, it collects dependency information by executing the test cases on the instrumented products. For the bytecode instrumentation, we used a Byte Code Engineering Library (BCEL),[4] which is the part of the Apache Jakarta project.

The Selector component selects regression test cases. For changes made to the common part of the source code, it searches

[4] http://commons.apache.org/proper/commons-bcel/.

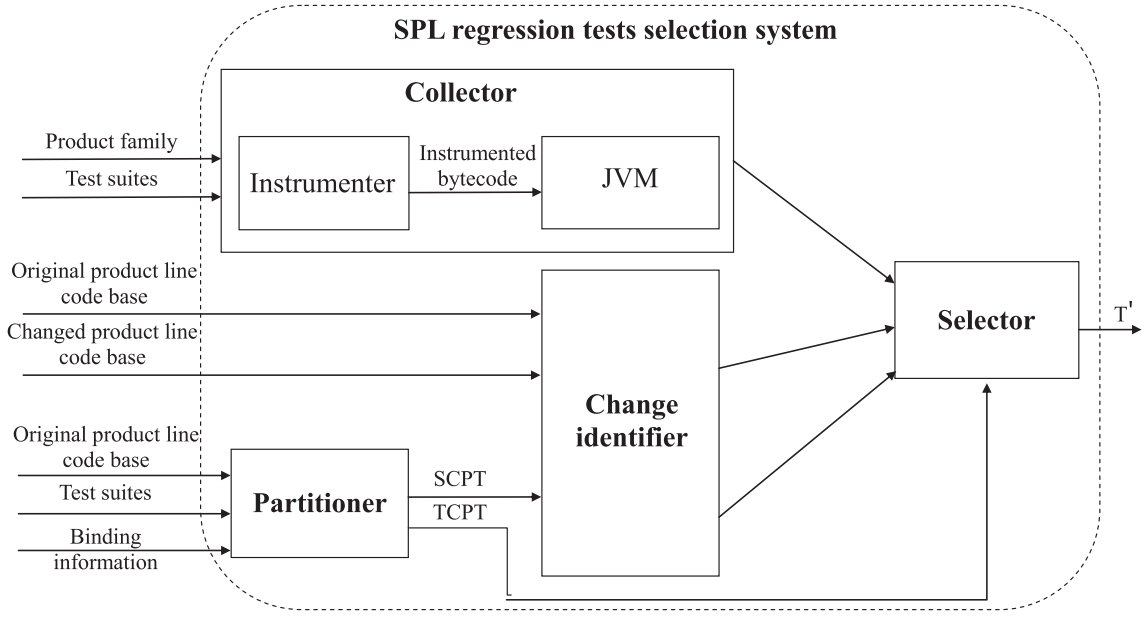


Fig. 11. Architecture of SRTST.

```
@Test
public void test04()  throws Throwable  {
    GameObject gameObject0 = new GameObject();
    gameObject0.id = 33;
    Vector<Scrollbar> vector0 = new Vector<Scrollbar>();
    vector0.setSize(565);
    // Undeclared exception!
    try {
      gameObject0.stossenGegen(vector0);
      fail("Expecting exception: NullPointerException");

    } catch(NullPointerException e) {
       assertThrownBy("GameObject", e);
    }
}
```

Fig. 12. A test case for the TankWar SPL.

Table 3
Subject product lines used in the experiments.

Subject SPL	SLOC	$\|P_{SPL}\|$ (#sp)	Com.	$\|T_{SPL}\|$ (Cov.)
MobileMedia	3196	4(11)	44.2%	1139(85.8%)
TankWar	4845	5(17)	62.9%	782(71.5%)
Prevayler	5109	3(4)	72.5%	1306(67.8%)
		5(6)	72.5%	2543(67.9%)
MobileRSS	16,664	3(5)	93.1%	3247(66.5%)
Reader		5(16)	93.1%	5383(66.3%)
Lampiro	30,158	4(10)	98.8%	6236(48.1%)
		6(16)	98.6%	9346(49.3%)
BerkeleyDB	44,994	3(4)	69.3%	10,297(57.2%)
		5(16)	69.3%	18,407(56.0%)
		7(31)	69.3%	26,290(59.3%)

the test classes dependent on the changed classes from dependency information. For changes made to the variable part of the source code, it searches the test cases affected by the changed spaces from the TCPT.

7. Evaluation

In this section, we evaluate our method using SRTST that was introduced in Section 6. The main objective of our evaluation is to show that in the context of the SPL, our method is effective in reducing the cost of regression testing. Section 7.1 describes our subjects and our experimental process in detail. Sections 7.2 presents the experimental results. Section 7.3 discusses our experimental results and Section 7.4 discusses the threats that can affect the validity of the results.

7.1. The experimental setup

7.1.1. Subject product lines, tests and versions

SPL2go is an open-source product line repository that is commonly used in SPL research (Angerer et al., 2017; Bodden et al., 2013; Kim et al., 2017; Krüger et al., 2018; Lopez-Herrejon et al., 2014). It has 40 SPL products (accessed August 2019). However, most of the projects are rather small (i.e., less than 3000 lines of product line code base). Excluding such small products and some projects that did not work due to errors, we selected six SPL projects, which are all implemented in Java. Table 3 describes the projects.

Table 3 lists the subjects and, for each of them, the number of source lines counted by LocMetrics[5] (SLOC), the number of features (#ftr), the number of products ($|P_{SPL}|$) with the number of spaces (#sp) in parentheses, the percentage of the source code common to all the products (Com.) and the number of test cases ($|T_{SPL}|$) with the percentage of the methods covered by the test cases (Cov.) in parentheses. Because the test coverage of a product line code base cannot be directly measured before its instantiation, we measured the test coverage of each planned product and averaged them. TankWar and BerkeleyDB were developed by the FeatureIDE tool and the other subjects were developed by the CIDE tool. We confirmed that all the subjects were generative product lines and well developed. For TankWar, their product families have been defined by their developers, but for the others, product families have not been defined and so we created product families with different numbers of products by randomly selecting their features. We confirmed that all the created product families had widely different sets of features.

[5] http://www.locmetrics.com/.

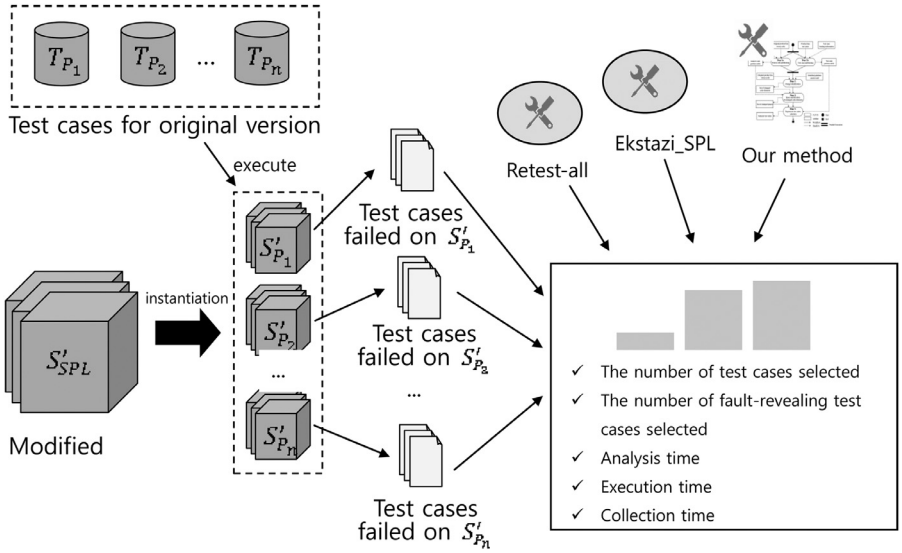


Fig. 13. Process of our experiments.

Our subject product lines had no test cases. For this reason, we generated product line test cases for them using the EvoSuite (Fraser and Arcuri, 2011) (with default settings), which is a search-based automatic test case generation tool. To use EvoSuite, a source code must be compiled. However, because the code bases of our subject product lines could not be compiled before instantiation, we could not directly generate the product line test cases from the product line code base. For this reason, for each subject, we first instantiated the product line code base to products of the product family and generated test cases for each product. Then we put these test cases in a repository and removed invalid test cases. Finally, we manually assigned each test case in the repository to its applicable products. Through these steps, we obtained product line test cases that include both test cases common to two or more products and test cases unique to a single product. Then we produced test cases for each product using these product line test cases. Because the coverage of test cases generated by EvoSuite is rather low and the test cases include only unit test cases, we also manually created unit and integration test cases to increase test coverage.

To generate fault-revealing test cases by injecting faults to the code bases of subject product lines, we created 50 modified versions of each subject product line by making 5 to 30 mutations generated by the PIT[6] mutation analysis tool. The mutations include addition/deletion of classes, methods and statements, replacing arithmetic/logical operators, negating conditional decisions and so on. The modified versions produced a set of products containing the injected faults. At this point, test cases that failed in the products were used as fault-revealing test cases in our experiments.

7.1.2. Methods for comparison

The goal of our method is to reduce the end-to-end time without missing any fault-revealing test cases. Therefore, to show the effectiveness of our method, our method should be compared with the methods that do not miss any fault-revealing test cases. We compare our method with the two methods described below, which do not miss any fault-revealing test cases and thus would lead to a fair comparison with respect to the end-to-end time.

The retest-all method selects all the product line test cases and they are executed on a single applicable product. Note that the selected test cases are not executed on every applicable products because we assumed that the test results of a test case can be reused over products where the test case is applicable. It always selects all the fault-revealing test cases. However, it requires much time and resources. This approach is used as a base line to assess our approach.

Ekstazi (Gligoric et al., 2015) is an RTS method for a single software system. For regression testing of SPL, it can be applied multiple times to products of a product family. We call this version of Ekstazi *SPL*. The Ekstazi_SPL selects test cases by iteratively applying Ekstazi to each product of a product family without any reuse. Therefore, it requires unnecessary efforts that can be avoided by reusing testing results of the products. The Ekstazi_SPL selects all the fault-revealing test cases because this method is just a repetition of Ekstazi over products of a product family and Ekstazi selects all the fault-revealing test cases (Gligoric et al., 2015).

7.1.3. Experimental design

We compare our method with the two methods introduced in Section 7.1.2). Our experimental process is depicted in Fig. 13. For each subject product line, we first produce planned products (S'_{P_1}, S'_{P_2}, ..., S'_{P_n}) of the modified versions (S'_{SPL}) created with the PIT tool. Then we execute test cases (T_{P_1}, T_{P_2}, ..., T_{P_n}) on the planned products. At this point, the test cases that fail or are not runnable on each product are the fault-revealing test cases of the product and should be selected for regression testing. Then we obtain sets of test cases selected by our method and the other two methods and execute them. Finally, we count the test cases selected by each method and measure the end-to-end time of each method. The end-to-end time is the total time to perform a RTS method (i.e., the sum of time to perform, for our method, *P*, *CC*, *A*, *E* and *C* phases, for the Ekstazi_SPL, *A*, *E* and *C* phases and for the retest-all method, *E* phase) and reducing the end-to-end time is important because it represents the overall cost of regression testing. This experiment was performed on Windows 7 with an Intel Core i7 CPU (3.40 GHz) and 12 GB RAM and we uploaded the experimental materials in the website: https://github.com/psjung/SRTST_experiments.

7.2. Results

The retest-all method, the Ekstazi_SPL and our method always included all the fault-revealing test cases, which indicates that they

[6] http://pitest.org/.

Table 4
Measurements for three methods.

Subject SPL	Our method				Ekstazi_SPL			Retest-all	
	#Sel	(*P*+CI)(s)	*A*(s)	(*E*+*C*)(s)	#Sel	*A*(s)	(*E*+*C*)(s)	#Sel	*E*(s)
MobileMedia(4)	384.6	0.11	0.26	1.57	455.9	0.31	2.39	3603	6.94
TankWar(5)	219.4	0.10	0.21	1.76	269.9	0.34	3.72	782	2.57
Prevayler(3)	381.3	0.10	0.52	4.21	261.1	0.70	5.17	1306	9.94
Prevayler(5)	798.6	0.15	0.72	10.60	594.7	1.03	12.43	2543	23.18
MobileRSSReader(3)	761.2	0.15	0.39	5.93	759.7	0.43	9.27	3247	11.07
MobileRSSReader(5)	1234.8	0.18	1.78	8.79	1258.0	1.86	17.13	5383	22.40
Lampiro(4)	1015.3	0.16	1.10	32.42	1009.1	1.13	45.63	6236	128.31
Lampiro(6)	1502.3	0.30	1.90	43.21	1520.7	2.02	68.82	9346	187.95
BerkeleyDB(3)	2513.1	0.18	0.95	12.89	2613.9	1.08	15.38	10,297	22.96
BerkeleyDB(5)	4419.4	0.31	1.94	23.70	5969.2	2.39	32.68	18,407	46.78
BerkeleyDB(7)	5501.5	0.38	2.81	25.08	8417.4	3.48	36.68	26,290	55.70
Average	1702.8	0.19	1.14	15.47	2102.7	1.34	22.66	7382.7	47.07

always select all the test cases affected by the source code change. However, regarding the test case reduction effect and the end-to-end time reduction, they had different results. Table 4 presents the average #Sel and the average time to perform the *P*, *CC*, *A*, *E* or *C* phases resulting from the application of the three methods to the eleven product families of our subject product lines. Because the *C* phase is performed together with the *E* phase, we presented the sum $(E+C)$ of the time to perform the two phases. From Table 4, the following observations can be made.

First, our method reduces more test cases for a retest and a longer end-to-end time to perform regression test selection compared to the Ekstazi_SPL. Our method reduced, on average, 76.9% (= (1–1702.8/7382.7) × 100%) of whole test cases and 64.3% (= (1-(0.19+1.14+15.47)/47.07) × 100%) of the time to perform the retest-all method. On the other hand, the Ekstazi_SPL reduced, on average, 71.5% (= (1–2102.7/7382.7) × 100%) of all the test cases and 49.0% (= (1-(1.34+22.66)/47.07) × 100%) of the time to perform the retest-all method.

Second, the overhead for additional phases (i.e., the *P* and *CC* phases) of our method, which is not required for the Ekstazi_SPL, is rather small compared to the end-to-end time. In our subject product families, (*P*+CC) phases took only by 0.1 ∼ 0.38 s while the end-to-end time by 1.94 ∼ 45.41 s.

Third, in our method and the Ekstazi_SPL, the time to perform the *A* phase is very short compared to the time to perform the *E* and *C* phases. This result is the same as the experimental result from Ekstazi as shown in Vasic et al. (2017). In our method, on average, the *A* phase takes 1.14 s while the $(C+E)$ phases takes 15.47 s. Likewise, in the case of the Ekstazi_SPL, on average, the *A* phase takes 1.34 s while the $(C+E)$ phases takes 22.66 s. This suggests that the end-to-end time for regression testing is more sensitive to the *E* and *C* phases than the *A* phase and so for cost reduction, it is necessary to reduce the time to perform the $(E+C)$ phases rather than the *A* phase.

To show the efficiency of our method, we compared our method with the Ekstazi_SPL with regard to #Sel and the end-to-end time. Table 5 presents the average percentages of the number of product line test cases selected (#Sel) and the end-to-end time resulting from our method and the Ekstazi_SPL on the subject product families.

The results confirmed that our method reduced, on average, more end-to-end time than the Ekstazi_SPL on all the subject product lines. Our method reduced more time required for the *A* phase and the $(E+C)$ phases than the Ekstazi_SPL, which resulted in a reduction of the end-to-end time by 14.8% ∼ 49.1%. In particular, for Prevayler(3), Prevayler(5), MobileRSSReader(3) and Lampiro(4), even though our method reduced fewer test cases than the Ekstazi_SPL, our method saved more end-to-end time than the Ekstazi_SPL. These results indicate that our method outperforms the Ekstazi_SPL by reducing unnecessary repetitions of the *AEC* phases and in-depth analysis effort of source code and test cases.

The overall distributions for #Sel and the end-to-end time demonstrate our method reduces more of the end-to-end time than the Ekstazi_SPL regardless of test cases reduction effect (cf. Appendix A). In addition, the two-sided Mann-Whitney *U* test and Cliff's Delta effect size demonstrate our method statistically significantly better than the Ekstazi_SPL in terms of the end-to-end time reduction (cf. Appendix B).

7.3. Discussion

7.3.1. Two factors to reduce more test cases

Our method can reduce more test cases when (1) a product family is partitioned into a large number of spaces (according to our experiments, more than 10 spaces) and (2) the Com. value of a product family is not very high (according to our experiments, below 93.1%). Regarding the first factor, in Table 5, in the case of the BerkeleyDB(7), which is partitioned into 31 spaces, our method reduced, on average, 34.6% more test cases than those selected by the Ekstazi_SPL. However in case of the BerkeleyDB(3), which is partitioned into only 4 spaces, our method reduced, on average, only 3.9% more test cases than those selected by the Ekstazi_SPL. On the other hand, our method reduced, on average, fewer test cases than the Ekstazi_SPL on product families that are partitioned into a small number of spaces. In the case of the Prevayler(3), Prevayler(5) and MobileRSSReader(3), our method reduced, respectively, 46.0%, 34.3% and 0.2% fewer test cases than the Ekstazi_SPL, on average. As presented in Table 3, the product line code base of the Prevayler(3), Prevayler(5) and MobileRSSReader(3) was partitioned into few spaces (i.e., respectively, 4, 6 and 5) due to the small differences among products. Because this made the size of their spaces large and the selection granularity of our method bigger than that of the Ekstazi_SPL (i.e., class granularity), our method selected, on average, more test cases than the Ekstazi_SPL. Regarding the second factor, our method is competitive with the Ekstazi_SPL on SPLs that have high value of Com. When a change is made to the common part of the source code, our method and the Ekstazi_SPL select the same set of test cases because, in that case, our method is just an Ekstazi_SPL without unnecessary repetitions. For this reason, the test cases selected from our method and the Ekstazi_SPL include a large number of the same test cases. As presented in Table 3, MobileRSSReader(3), MobileRSSReader(5), Lampiro(4) and Lampiro(6) have a high value of Com., so on these product families, our method selected a very similar number of test cases with the Ekstazi_SPL. In Appendix B-(a), we provide the boxplots depicting the #Sel values resulting from our method and the Ekstazi_SPL.

Table 5
Comparison of our method and the ekstazi_SPL.

Subject SPL	#sp	#Sel			End-to-end time		
		Our method	Ekstazi_SPL	Savings	Our method	Ekstazi_SPL	Savings
MobileMedia(4)	11	10.7%	12.7%	15.6%	28.0%	39.0%	28.1%
TankWar(5)	17	28.1%	34.5%	18.7%	80.5%	158.0%	49.1%
Prevayler(3)	4	29.2%	20.0%	−46.0%	48.6%	59.1%	17.1%
Prevayler(5)	6	31.4%	23.4%	−34.3%	49.5%	58.1%	14.8%
MobileRSSReader(3)	5	23.4%	23.4%	−0.2%	58.4%	87.6%	33.3%
MobileRSSReader(5)	16	22.9%	23.4%	1.8%	48.0%	84.8%	43.4%
Lampiro(4)	10	16.3%	16.2%	−0.6%	26.2%	36.4%	28.0%
Lampiro(6)	16	16.1%	16.3%	1.2%	24.2%	37.7%	35.9%
BerkeleyDB(3)	4	24.4%	25.4%	3.9%	61.1%	71.7%	14.8%
BerkeleyDB(5)	16	24.0%	32.4%	26.0%	55.5%	75.0%	26.0%
BerkeleyDB(7)	31	20.9%	32.0%	34.6%	50.8%	72.1%	29.6%

7.3.2. End-to-end time reduction

A main goal of regression testing is to increase the fault detection rate for a changed program and decrease the cost required for a retest (Yoo and Harman, 2012). Therefore, it is reasonable that the test case reduction effect is sacrificed for cost reduction without a reduction in the fault detection rate (Gligoric et al., 2015). Our method sacrifices the test case reduction effect depending on the Com. and #sp values of a product family. Nevertheless, we can say our method is more effective than the Ekstazi_SPL because it reduces more end-to-end time than the Ekstazi_SPL without missing fault-revealing test cases.

7.3.3. Testing of new products

Our method retests only the affected planned products because retesting all the affected (planned and unplanned) products whenever a change is made would be very costly. Our method does not retest the unplanned products until they are planned and produced, which makes SPL regression testing scalable. If previously unplanned products are newly added as planned products, this can be easily handled with our method by updating SCPT and TCPT with source code and test cases for the new planned products and collecting dependency information for them.

7.3.4. Using other programming languages

Our method currently works only for the SPL systems written in Java because in the *c_A* phase, our method uses a RTS method for single software systems and, currently in the work of this paper, we applied Ekstazi, an RTS method for Java software. However, if, instead of Ekstazi, RTS methods for other programming languages are used in our method, then our method will also work for other languages. Using other RTS methods for our method would be realized straightforwardly because the *c_A* phase is just a repetition of an RTS method for a single software system, which is very simple as can be found from Section 4.1.

7.3.5. Applicability to configurable systems

A configurable system is a system such that it has various configuration options to control the system's behavior and by setting values for the configuration options an instance of the system is produced (Qu et al., 2012). So a configurable system has much similarity with a product line in that a set of values for configuration options forms binding information and its instance can be viewed as a product. Accordingly, our method is applicable also to configurable systems because instances of a configurable system have common parts and variable parts among them and so their source code can be partitioned into spaces based on a set of values for configuration options. We obtained the Elevator configurable system from SIR repository and then partitioned its source code. As a result, 6 spaces were obtained and they could be used for RTS. This result indicates that our method can be applied to configurable systems.

7.3.6. Impact of the Deterministic product line test run assumption

A test case execution on a single product can be nondeterministic. So, when a change is made to a certain common code part that can be potentially covered by a certain execution of a test case but, for the test execution, dependencies of the test case have not yet been collected, our method may miss a fault-revealing test case. In such case, by collecting as many dependencies of test cases as possible through a large number of test executions, we can reduce the probability of missing fault-revealing test cases. Also, a test case can have different executions on different products. For example, for products P1 and P2, if a test case is reused for both products to test two different code parts that are specific to P1 and P2, respectively, the test case has different executions on the two products. In this case, when a change is made to one of the code parts, our method will not select this change-relevant test case and so can miss a fault-revealing test case. However, such a test case should be avoided because the test cases that have multiple executions over multiple products make change impact analysis for SPL regression testing very complex. To avoid this problem, such test cases should be divided into several test cases that have a single execution.

7.3.7. Impact of the well-developed product line code base assumption

When a product line code base includes duplicated code fragments and a test case is reused for the code fragments, the test cases partition table will be vulnerable to the change to the code fragments. For example, if one of the duplicated code fragments is changed alone, the consistency between test cases and the test cases partition table will be lost. To address this problem, an automated way to update the test cases partition table should be developed. Moreover, if the code base of a product line has invalid use relationships, the product line can produce invalid products. In this case, the test cases selected from our method can be invalid for the products. So our method should be used after fixing all invalid use relationships in the product line code base.

7.4. Threats to validity

7.4.1. Internal validity

SRTST may contain defects. To overcome this threat, we reviewed its code, tested it on several small product lines, and manually inspected the results.

7.4.2. External validity

(Validity of subject product lines) First, our subject product lines may not be representative of real product lines. To mitigate this

threat, we selected open-source product lines that were recently used in other SPL research as experimental objects (Angerer et al., 2017; Bodden et al., 2013; Kim et al., 2017; Krüger et al., 2018; Lopez-Herrejon et al., 2014). They vary in sizes, percentage of commonality, number of features, and application domain. We also experimented with product families with different numbers of products. *(Validity of test case and fault)* Second, the test cases and faults used in our experiments may not be representative of real ones. To address this threat, in terms of test case generation, we used EvoSuite, which is the state-of-the-art tool that obtained the highest overall score in coverage and fault detection in a competition with other tools (Fraser and Arcuri, 2018). In terms of fault generation, a mutation tool has been commonly used for generating faults as in previous testing research (Qu et al., 2012; Romano et al., 2018), so we used the PIT mutation tool to inject faults because, as reported in Just et al. (2014), mutation faults are close to real faults and are suitable for software testing experiments. *(Validity of product line test cases)* Third, to generate a set of product line test cases, we produced a product family and then generated test cases for each product of the family because tools for automated SPL test case generation do not exist and the test cases of our subject product lines could not be directly generated by EvoSuite. However, the test cases generated for each product may not include test cases common to two or more products. Then they cannot be used as product line test cases. To overcome this threat, we carefully classified the test cases generated for each product into test cases that can be reused for other products and test cases unique to a single product, and afterwards we manually assigned each test case to the relevant products. Through this process, we obtained the test cases that are common to two or more products and test cases that are specific to a single product, and also produced test cases for each product of a product family.

8. Related work

Regression testing of a single software product has been extensively investigated, compared, and evaluated as the surveys in Engström et al. (2010), Khan et al. (2018), Singh et al. (2012) and Yoo and Harman (2012) show. However, there is no evidence that these techniques still work in the context of an SPL as pointed out in Wang et al. (2016). Engström and Runeson (2010) conducted a qualitative survey to investigate industrial practice and concluded that test automation is an issue that requires much improvement, and test case selection for continuous regression testing is challenging. For these reasons, a test case selection and its automation in the context of an SPL is an important research topic.

Section 8.1 classifies regression testing approaches for SPL into studies on test case prioritization, minimization and optimization and discusses them. Section 8.2 discusses in detail the studies on test case selection. Sections 8.3 and 8.4 discuss the studies on combinatorial product selection and test case reuse for testing of new products, respectively.

8.1. Regression testing of a software product line

Several researchers suggested approaches for regression test prioritization (Al-Hajjaji et al., 2016b; Devroey et al., 2014; Ensan et al., 2011; Henard et al., 2014; Lachmann et al., 2015; Markiegi et al., 2017; Parejo et al., 2016; Sánchez et al., 2014; Wang et al., 2014), reduction (Baller et al., 2014; Hawkey et al., 2012; Wang et al., 2015), optimization (Marijan et al., 2019; Marijan et al., 2017) and selection (covered in Section 8.2) in the context of SPL.

8.1.1. Test prioritization

Al-Hajjaji et al. (2016b) and Henard et al. (2014) proposed similarity-based product prioritization techniques, which prioritize products of a product line based on similarity of their selected features. They assigned a high priority to the products that are dissimilar to previously tested products. Devroey et al. (2014) suggested a behavior-based statistical product prioritization algorithm. The algorithm prioritizes products of a product line according to the probability that their behaviors happen. While these approaches prioritize products of a product line, some approaches prioritize test cases of a product line. Lachmann et al. (2015) proposed a delta-oriented test prioritization approach based on delta modeling. The approach computes regression deltas, which are the differences between products of a product family, and then prioritize test cases by analyzing the change impact using the regression deltas. Ensan et al. (2011) proposed a goal-oriented test prioritization approach. The approach first assigns priorities to features according to the degree of stakeholder's goal satisfaction and then prioritizes test cases based on the priorities. Wang et al. (2014), Parejo et al. (2016) and Markiegi et al. (2017) proposed search-based test prioritization approaches for SPLs. They addressed a multi-objective optimization problem for SPL test prioritization and evaluated their approaches with existing search based algorithms. Sánchez et al. (2014) conducted a comparison study with six test case prioritization criteria using 15 feature models. They concluded that the combination of the variability coverage and the cyclomatic complexity produces the best results in terms of average percentage of faults detected. The above approaches could be applied to RTS because test case selection can be viewed as just test case prioritization that deselects test cases that are below a particular threshold. However, test case prioritization assumes that all the test cases may be executed (Yoo and Harman, 2012) while test case selection does not execute all the test cases since it selects a subset of all the test cases to be rerun on a changed program.

8.1.2. Test minimization

Three approaches (Baller et al., 2014; Hawkey et al., 2012; Wang et al., 2015) for test case prioritization have been proposed in the context of SPL. Hawkey et al. (2012) classified test cases into obsolete test cases, reusable test cases and re-testable test cases and then minimized test cases for a product family using the classified test cases. Baller et al. (2014) proposed a multi-objective technique for test suite minimization. The objective function of the technique aims at minimizing the size of test suites and the execution cost. Wang et al. (2015) proposed a random-weighted genetic algorithm for test case minimization. The fitness function of the genetic algorithm optimizes pairwise coverage, fault detection capability, execution frequency, execution time and size of test suite. Test case selection is different from test case reduction in that the latter changes the existing set of test cases whereas test case selection selects a subset of existing test cases without change.

8.1.3. Test optimization

Marijan et al. (2019) and Marijan et al. (2017) suggested two approaches that optimize a test suite for a configurable system. One approach is called TITAN (Marijan et al., 2017). Once a regression test suite is selected, TITAN optimizes the test suite using both test minimization technique and test prioritization technique so as to achieve a high fault detection rate and a low test execution time. The other approach (Marijan et al., 2019) is a learning algorithm that reduces test redundancy. The algorithm minimizes the test execution time by avoiding unnecessary test executions using coverage metrics and a fault-detection history. These approaches are complementary to an RTS method for SPL because they can

optimize SPL regression testing by reducing the executions of test cases selected from the RTS method.

The existing SPL regression testing approaches share the same objective, which is to reduce cost of regression testing. However, for the reasons discussed above the techniques for test case prioritization, reduction and optimization cannot be directly applied to test case selection and therefore techniques for test selection were explored as discussed in the next section.

8.2. Test selection for regression testing of a software product line

Neto et al. (2010) presented an architecture based SPL regression testing framework at the integration level and defined its activities, steps, roles and artifacts. They could reduce testing effort through selection and prioritization of test cases on the basis of architectural similarities between products. However, their approach requires the intervention of human experts with much experience in software testing.

Engström (2010) suggested research directions based on two aspects of test selection in the context of an SPL: the test coverage of a product line and the selection of configuration to be executed for a retest. Engström said that a tool for visualizing the system test coverage for a product line would be a great help in managing the two aspects. Unfortunately, this research is not very useful to practitioners because no concrete procedure (i.e., algorithm or tool) for RTS was offered.

Lity et al. (2018) applied a delta modeling technique and a model slicing technique to SPL regression testing. Given the state machines of each product of a product family, deltas between them are reflected in each state machine. When a change occurs, a set of slices, which consist of states and transitions affected by the change, is derived from the state machines. Regression test suites are selected by applying retest coverage criteria to the slices. A limitation of this approach is that when state machines have not been strictly managed, it can miss faults. Our method and that of Lity et al. (2018) have significant differences. First, our method and that of Lity et al. (2018) require different types of input and produces different kinds of outputs. The central input to our method is source code whereas the central input to the method of Lity et al. (2018) is a set of state machines. Output from our method is a set of code-level regression test cases whereas output from the method of Lity et al. (2018) is a set of paths (i.e., alternating sequences of states and transitions to be retested) through state machines. Second, the research scopes of our method and that of Lity et al. (2018) are different. Our method covers all the phases of RTS (i.e., the *AEC* phases) whereas that of Lity et al. (2018) covers only the *AE* phases and generation of new test cases. The method of Lity et al. (2018) assumes that traceability between test cases and state machines has been constructed. So if the assumption is removed so as to make it work with the same kinds inputs and outputs of our method, then the *C* phase should be manually performed incurring extensive human efforts because there is no currently known automatic way that evolves state machines or constructs traceability between code-level test cases and state machines. Third, our method uses a coarse-grained selection for saving the cost of the *A* and *C* phases while the method of Lity et al. (2018) uses a more fine-grained selection in order to minimize the number of test cases for a retest. Finally, the goal of our method is to reduce the overall time for the *AEC* phases of selection procedure without missing fault-revealing test cases whereas that of Lity et al. (2018) is to achieve a high fault detection rate.

As can be seen above, there have been few studies on RTS for SPLs but they have critical limitations. We suggested an automated code-based RTS method for SPLs that overcomes the existing limitations.

In our previous work (Jung et al., 2018), we proposed an RTS method for changes made to variable part of an SPL, and showed that it can reduce test cases for a retest without missing any fault-revealing test cases. However, because it does not handle changes made to the common parts of an SPL, it is limited that when a change is made to the common part of an SPL, all test cases are selected for a retest (i.e., retest-all). In addition, the previous work just focused on reducing the number of test cases even though reducing the overall testing time is a practical goal for regression testing. To overcome these limitations, this paper extended our previous work (Jung et al., 2018) to include the steps to handle the common part of an SPL and present additional evaluation results.

8.3. Combinatorial product selection for software product line testing

Combinatorial techniques such as ICPL (Johansen et al., 2012), CASA (Garvin et al., 2011), IncLing (Al-Hajjaji et al., 2016a) have been proposed to perform *t*-wise sampling for a product line. Their common insight is that most of faults are caused by interactions among a fixed number of at most *t* features. They systematically reduce the number of products under test by generating a minimal set of products so that at least one product covers *t*-wise interaction. Ferreira et al. (2017) proposed a multi-objective product sampling technique, which selects a set of products to test considering four objectives: the number of products, pair-wise coverage, mutation score and dissimilarity of products. Hervieu et al. (2016) proposed a constraint programming approach called PACOGEN. PACOGEN generates a minimized set of valid configurations that ensure 100% pair-wise feature coverage without violations of constraints.

The main goal of these techniques is to obtain a subset from a set of all possible products such that in the products of the subset feature interaction faults are most likely to occur. So they were used for test case generation (Jia et al., 2015; Lin et al., 2015; Qi et al., 2016; Yu et al., 2013) and prioritization (Al-Hajjaji et al., 2016b; Henard et al., 2014). However, these techniques were not designed to take changes to a product line into account and therefore if these techniques are applied to test case selection for regression testing, they will miss change-relevant test cases that expose faults. Furthermore, the combinatorial techniques not only consider planned products but also unplanned products that need not be produced. But, in practice, only a subset of all possible products is actually planned to be produced as a product family. For this reason, this paper focused on testing of planned products.

8.4. Reuse of test cases for testing new product of a software product line

Some approaches (Li et al., 2018; Lochau et al., 2012; Wang et al., 2016; Wang et al., 2017; Xu et al., 2013) select a subset of existing test cases to generate test cases for a new product (i.e., a product not tested in a product family). They reuse the test cases for the existing products by analyzing differences between products of a product family. However, they just focused on obtaining reusable test cases for a new product and did not deal with selective retest (Rothermel and Harrold, 1996). Differences between two products of a product family can be considered as regression changes, which can be used for RTS. However, they are changes made to a single product but not to a product family. Therefore, reusing existing test cases to generate test cases for a new product is different from RTS for SPL.

9. Conclusion

In this paper, we proposed a method of automated code-based regression test selection for software product lines. When changes are made to product line code base, our method selects regression

tests by leaving out change-irrelevant test cases based on the commonality and variability of the product family. Our method individually dealt with changes made to the common part of the source code and to the variable part of the source code. For changes made to the common part of the source code, our method reduced unnecessary repetitions of a selection procedure over products of a product family and for changes made to the variable part of the source code, our method excluded an internal analysis of the source code or a coverage analysis of test cases. Our method does not require requirements specification, architecture or traceabilities for test cases. Therefore, it can be applied even to SPLs where these artifacts are not well managed enough to select test cases for a retest. In our experimental evaluation, we compared our method with the Ekstazi_SPL using six SPLs. Our method reduced the end-to-end time to perform regression testing by 14.8% ∼ 49.1% on our subject product families without missing any fault-revealing test cases.

We are currently investigating the following directions for future work. First, we plan to extend our method so that it always selects all the fault-revealing test cases even without assuming that a product line code base is well-developed. Second, our research in this paper considered only generative product lines. We will extend our method to make it work for non-generative product lines too. Third, in order to widen the applicability of our method, we will expand our method to cover regression test cases with non-deterministic test executions. Finally, we will apply our method to larger scale SPLs from the industry.

Acknowledgements

This research was supported by Next-Generation Information Computing Development Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Science, ICT (NRF-2017M3C4A7066210) and by Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education (2017R1D1A3B03028609).

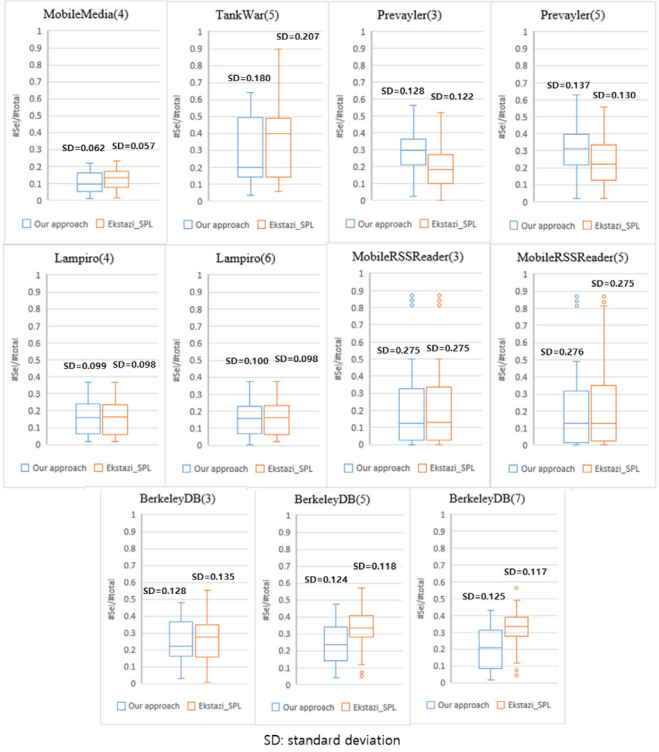
Appendix A. Boxplots for each subject product family resulting from our method and the Ekstazi_SPL

This appendix augments our experimental results in Section 7.2. Fig. 14 presents the boxplots for #Sel (a) and the end-to-end time (b) grouped by method and subject product family.

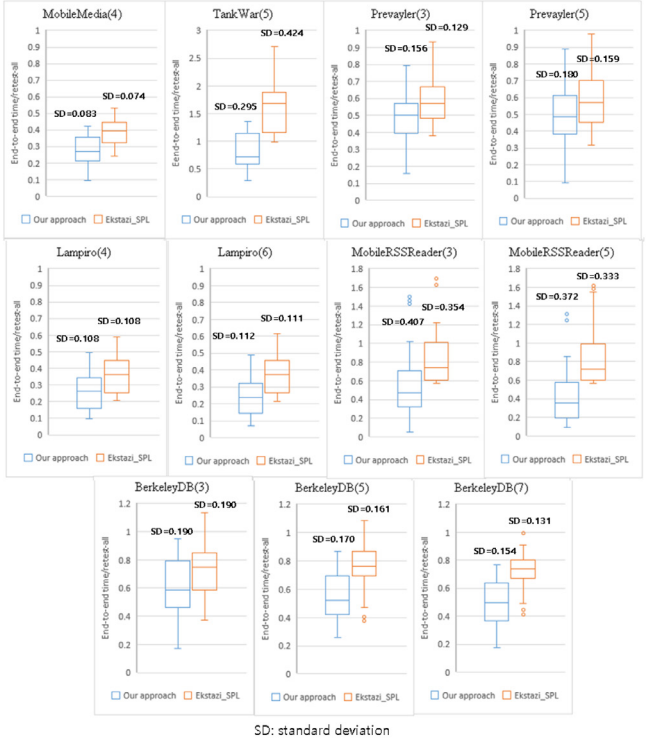
The boxplots for #Sel show that for MobileRSSReader(3), MobileRSSReader(5), Lampiro(4) and Lampiro(6), the boxplots of our method and the Ekstazi_SPL are located in similar positions; for those of Prevayler(3) and Prevayler(5), the boxplots of our method are located in higher positions than those of the Ekstazi_SPL; for the others, the boxplots of our method are located in lower positions than those of the Ekstazi_SPL. On the other hand, the boxplots for the end-to-end time show that the boxplot of our method is located in a lower position than that of the Ekstazi_SPL for all the subject product lines. This confirms that as stated in Section 7.2, our method saves more of the end-to-end time than the Ekstazi_SPL does regardless of test case reduction effect, which indicates that for cost reduction of regression testing, our method statistically outperforms the Ekstazi_SPL on all the subject product families.

Appendix B. Statistical analysis of measurements resulting from our method and the Ekstazi_SPL

This appendix contains the result of the statistical analysis. To analyze differences between the results obtained from our method and the Ekstazi_SPL, we conducted a two-sided Mann-Whitney *U* test (McDonald, 2009), which is a nonparametric statistical test. The null hypothesis is that two samples come from the same population. We chose a significance level of 0.05, so if the p-value is



(a) The number of test cases selected for each subject product family



(b) The end-to-end time for each subject product family

Fig. 14. Distribution of #Sel and the end-to-end time resulting from our method and the Ekstazi_SPL.

Table 6
P-value obtained by Mann-Whitney *U* test and Cliff's Delta effect size.

SubjectSPL	#Sel		End-to-end time	
	p-value	Effect size	p-value	Effect size
MobileMedia(4)	0.067	Small (−0.212)	<0.001	Large (−0.648)
TankWar(5)	0.368	Negligible (−0.105)	<0.001	Large (−0.818)
Prevayler(3)	<0.001	Medium (0.422)	<0.001	Medium (−0.375)
Prevayler(5)	0.003	Medium (0.34)	0.026	Small (−0.258)
MobileRSSReader(3)	0.968	Negligible (0.005)	<0.001	Large (−0.591)
MobileRSSReader(5)	0.772	Negligible (−0.034)	<0.001	Large (−0.648)
Lampiro(4)	0.984	Negligible (−0.003)	<0.001	Medium (−0.472)
Lampiro(6)	0.849	Negligible (−0.022)	<0.001	Large (−0.585)
BerkeleyDB(3)	0.889	Negligible (−0.017)	0.008	Small (−0.316)
BerkeleyDB(5)	<0.001	Medium (−0.37)	<0.001	Large (−0.58)
BerkeleyDB(7)	<0.001	Large (−0.478)	<0.001	Large (−0.72)

lower than 0.05, we reject the null hypothesis, which means that the results from our method and the Ekstazi_SPL are significantly different. To reduce Type-I-error (i.e., α=0.05), we applied the Bonferroni correction method (Dunn, 1961).

The Mann–Whitney *U* test just checks the presence of a difference between two samples, but it does not provide the magnitude of the difference. To quantify this difference between the results obtained from our method and the Ekstazi_SPL, we measured the Cliff's Delta (δ) effect size (Cliff, 1993). The magnitude of the effect size is negligible if $|\delta| < 0.147$, small if $0.147 \leq |\delta| < 0.33$, medium if $0.33 \leq |\delta| < 0.474$ and large if $|\delta| \geq 0.474$. A high magnitude of the effect size implies that the probability that one method obtains a better score than the other method over many trials is high. A sign of δ value is used to determine which method is statistically better than the other. In our experiment, for both #Sel and the end-to-end time, a positive value means that our method is less effective than the Ekstazi_SPL and a negative value means that our method is more effective than the Ekstazi_SPL in terms of the probability that our method obtains a better score than the Ekstazi_SPL.

Table 6 presents the results of the Mann-Whitney *U* test and the Cliff's Delta effect size values on the subject product families. Regarding #Sel, our method reduced more test cases than the Ekstazi_SPL on the seven subject product families. However, in most cases, they had a negligible effect size or p-values more than 0.001. This indicates that test case reduction effect of our method is not statistically better than that of the Ekstazi_SPL. On the other hand, regarding the end-to-end time, our method reduced a significant amount of end-to-end time on all the subject product lines. In particular, nine out of eleven product families have p-values lower than 0.001 and none of them has a negligible effect size. This indicates that our method is statistically significantly better than the Ekstazi_SPL for cost reduction.

References

Al-Hajjaji, M., Krieter, S., Thüm, T., Lochau, M., Saake, G., 2016a. IncLing: efficient product-line testing using incremental pairwise sampling. In: ACM SIGPLAN Notices, 52. ACM, pp. 144–155.

Al-Hajjaji, M., Thüm, T., Lochau, M., Meinicke, J., Saake, G., 2016b. Effective product-line testing using similarity-based product prioritization. Softw. Syst. Model. 1–23.

Angerer, F., Grünbacher, P., Prähofer, H., Linsbauer, L., 2017. An experiment comparing lifted and delayed variability-aware program analysis. In: Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 148–158.

Baker, N.H., Kasirun, Z.M., Salleh, N., 2015. Feature extraction approaches from natural language requirements for reuse in software product lines: a systematic literature review. J. Syst. Softw. 106, 132–149.

Baller, H., Lity, S., Lochau, M., Schaefer, I., 2014. Multi-objective test suite optimization for incremental product family testing. In: Proceedings of the IEEE Seventh International Conference on Software Testing, Verification and Validation, pp. 303–312.

Batory, D., 2005. Feature models, grammars, and propositional formulas. In: Proceedings of the International Conference on Software Product Lines, pp. 7–20.

Bodden, E., Tolêdo, T., Ribeiro, M., Brabrand, C., Borba, P., Mezini, M., 2013. Spl lift: statically analyzing software product lines in minutes instead of years. ACM SIGPLAN Not. 48 (6), 355–364.

Cliff, N., 1993. Dominance statistics: ordinal analyses to answer ordinal questions. Psychol. Bull. 114 (3), 494.

Devroey, X., Perrouin, G., Cordy, M., Schobbens, P.-Y., Legay, A., Heymans, P., 2014. Towards statistical prioritization for software product lines testing. In: Proceedings of the Eighth International Workshop on Variability Modelling of Software-Intensive Systems, p. 10.

Dunn, O.J., 1961. Multiple comparisons among means. J. Am. Stat. Assoc. 56 (293), 52–64.

Engström, E., Runeson, P., 2010. A qualitative survey of regression testing practices. In: Proceedings of the International Conference on Product Focused Software Process Improvement, pp. 3–16.

Engström, E., Runeson, P., Skoglund, M., 2010. A systematic review on regression test selection techniques. Inf. Softw. Technol. 52 (1), 14–30.

Engström, E., Runeson, P., 2011. Software product line testing–a systematic mapping study. Inf. Softw. Technol. 53 (1), 2–13.

Engström, E., 2010. Regression test selection and product line system testing. In: Proceedings of the Third International Conference on Software Testing, Verification and Validation, pp. 512–515.

Ensan, A., Bagheri, E., Asadi, M., Gasevic, D., Biletskiy, Y., 2011. Goal-oriented test case selection and prioritization for product line feature models. In: Proceedings of the Information Technology: New Generations (ITNG), pp. 291–298.

Ferreira, T.N., Lima, J.A.P., Strickler, A., Kuk, J.N., Vergilio, S.R., Pozo, A., 2017. Hyper-Heuristic based product selection for software product line testing. IEEE Comput. Intell. Mag. 12 (2), 34–45.

Fraser, G., Arcuri, A., 2011. EvoSuite: automatic test suite generation for object-oriented software. In: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering, pp. 416–419.

Fraser, G., Arcuri, A., 2018. EvoSuite at the SBST 2018 tool competition. In: Proceedings of the 11th International Workshop on Search-Based Software Testing, pp. 34–37.

Garvin, B.J., Cohen, M.B., Dwyer, M.B., 2011. Evaluating improvements to a meta-heuristic search for constrained interaction testing. Empir. Softw. Eng. 16 (1), 61–102.

Gligoric, M., Eloussi, L., Marinov, D., 2015. Practical regression test selection with dynamic file dependencies. In: Proceedings of the International Symposium on Software Testing and Analysis, pp. 211–222.

Gregg, S.P., Albert, D.M., Clements, P., 2017. Product line engineering on the right side of the V. In: Proceedings of the 21st International Systems and Software Product Line Conference-Volume A, pp. 165–174.

Harrold, M.J., Orso, A., 2008. Retesting software during development and maintenance. Front. Softw. Maint. FoSM 2008, 99–108.

Haugen, Ø., Wąsowski, A., Czarnecki, K., 2013. CVL: common variability language. In: Proceedings of the 17th International Software Product Line Conference, p. 277.

Hawkey, K., Gao, Q., McAllister, M., Keselj, V., Gentleman, M., 2012. Test Cases Reduction in Software Produt Line Using Regression Testing M.S. thesis. Faculty of Computer Science, Dalhousie University.

Henard, C., Papadakis, M., Perrouin, G., Klein, J., Heymans, P., Le Traon, Y., 2014. Bypassing the combinatorial explosion: using similarity to generate and prioritize t-wise test configurations for software product lines. IEEE Trans. Softw. Eng. 1.

Hervieu, A., Marijan, D., Gotlieb, A., Baudry, B., 2016. Practical minimization of pairwise-covering test configurations using constraint programming. Inf. Softw. Technol. 71, 129–146.

Jia, Y., Cohen, M.B., Harman, M., Petke, J., 2015. Learning combinatorial interaction test generation strategies using hyperheuristic search. In: Proceedings of the 37th International Conference on Software Engineering-Volume 1, pp. 540–550.

Johansen, M.F., Haugen, Ø., Fleurey, F., 2012. An algorithm for generating t-wise covering arrays from large feature models. In: Proceedings of the 16th International Software Product Line Conference-Volume 1, pp. 46–55.

Jung, P., Kang, S., Lee, J., Park, T., 2018. Automated code-based test selection for software product line regression testing. In: Proceedings of the 25th Asia-Pacific Software Engineering Conference (APSEC), pp. 663–667.

Just, R., Jalali, D., Inozemtseva, L., Ernst, M.D., Holmes, R., Fraser, G., 2014. Are mutants a valid substitute for real faults in software testing? In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 654–665.
Khan, S.U.R., Lee, S.P., Javaid, N., Abdul, W., 2018. A systematic review on test suite reduction: approaches, experiment's quality evaluation, and guidelines. IEEE Access 6, 11816–11841.
Kim, J., Batory, D., Dig, D., 2017. Refactoring Java Software Product Lines. In: Proceedings of the 21st International Systems and Software Product Line Conference-Volume A, pp. 59–68.
Krüger, J., Al-Hajjaji, M., Leich, T., Saake, G., 2018. Mutation operators for feature-oriented software product lines. Softw. Test. Verif. Reliab. e1676.
Lachmann, R., Lity, S., Lischke, S., Beddig, S., Schulze, S., Schaefer, I., 2015. Delta-oriented test case prioritization for integration testing of software product lines. In: Proceedings of the 19th International Conference on Software Product Line, pp. 81–90.
Laguna, M.A., Crespo, Y., 2013. A systematic mapping study on software product line evolution: from legacy system reengineering to product line refactoring. Sci. Comput. Program. 78 (8), 1010–1034.
Lee, J., Kang, S., Lee, D., 2012. A survey on software product line testing. In: Proceedings of the 16th International Software Product Line Conference-Volume 1, pp. 31–40.
Li, X., Wong, W.E., Gao, R., Hu, L., Hosono, S., 2018. Genetic algorithm-based test generation for software product line with the integration of fault localization techniques. Empir. Softw. Eng. 23 (1), 1–51.
Lin, J., Luo, C., Cai, S., Su, K., Hao, D., Zhang, L., 2015. TCA: an efficient two-mode meta-heuristic algorithm for combinatorial test generation (t). In: Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 494–505.
Lity, S., Nieke, M., Thüm, T., Schaefer, I., 2018. Retest test selection for product-line regression testing of variants and versions of variants. J. Syst. Softw. 147, 46–63.
Lochau, M., Schaefer, Kamischke, I., J., Lity, S., 2012. Incremental model-based testing of delta-oriented software product lines. In: Proceedings of the International Conference on Tests and Proofs, pp. 67–82.
Lopez-Herrejon, R.E., Ferrer, J., Chicano, F., Egyed, A., Alba, E., 2014. Comparative analysis of classical multi-objective evolutionary algorithms and seeding strategies for pairwise testing of software product lines. In: Proceedings of the IEEE Congress on Evolutionary Computation (CEC), pp. 387–396.
Marijan, D., Gotlieb, A., Liaaen, M., 2019. A learning algorithm for optimizing continuous integration development and testing practice. Softw. Pract. Exp. 49 (2), 192–213.
Marijan, D., Liaaen, M., Gotlieb, A., Sen, S., Ieva, C., 2017. Titan: test suite optimization for highly configurable software. In: Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST), pp. 524–531.
Marimuthu, C., Chandrasekaran, K., 2017. Systematic studies in software product lines: a tertiary study. In: Proceedings of the 21st International Systems and Software Product Line Conference-Volume A, pp. 143–152.
Markiegi, U., Arrieta, A., Sagardui, G., Etxeberria, L., 2017. Search-based product line fault detection allocating test cases iteratively. In: Proceedings of the 21st International Systems and Software Product Line Conference-Volume A, pp. 123–132.
McDonald, J.H., 2009. Handbook of Biological Statistics, 2. sparky house publishing, Baltimore, MD.
Neto, P.A.d.M.S., do Carmo Machado, I., Cavalcanti, Y.C., de Almeida, E.S., Garcia, V.C., de Lemos Meira, S.R., 2010. A regression testing approach for software product lines architectures. In: Proceedings of the Fourth Brazilian Symposium on Software Components, Architectures and Reuse (SBCARS), pp. 41–50.
Neto, P.A.d.M.S., do Carmo Machado, I., McGregor, J.D., De Almeida, E.S., de Lemos Meira, S.R., 2011. A systematic mapping study of software product lines testing. Inf. Softw. Technol. 53 (5), 407–423.
Parejo, J.A., Sánchez, A.B., Segura, S., Ruiz-Cortés, A., Lopez-Herrejon, R.E., Egyed, A., 2016. Multi-objective test case prioritization in highly configurable systems: a case study. J. Syst. Softw. 122, 287–310.
Pohl, K., Böckle, G., van Der Linden, F.J., 2005. Software Product Line engineering: foundations, Principles and Techniques. Springer Science & Business Media.
Qi, R.-.Z., Wang, Z.-J., Li, S.-.Y., 2016. A parallel genetic algorithm based on spark for pairwise test suite generation. J. Comput. Sci. Technol. 31 (2), 417–427.
Qu, X., Acharya, M., Robinson, B., 2012. Configuration selection using code change impact analysis for regression testing. In: Proceedings of the 28th IEEE International Conference on Software Maintenance, pp. 129–138.
Rattan, D., Bhatia, R., Singh, M., 2013. Software clone detection: a systematic review. Inf. Softw. Technol. 55 (7), 1165–1199.
Romano, S., Scanniello, G., Antoniol, G., Marchetto, A., 2018. SPIRITuS: a SimPle Information Retrieval regressIon Test Selection approach. Inf. Softw. Technol. 99, 62–80.
Rothermel, G., Harrold, M.J., 1996. Analyzing regression test selection techniques. IEEE Trans. Softw. Eng. 22 (8), 529–551.
Sánchez, A.B., Segura, S., Ruiz-Cortés, A., 2014. A comparison of test case prioritization criteria for software product lines. In: Proceedings of the IEEE Seventh International Conference on Software Testing, Verification and Validation (ICST), pp. 41–50.
Singh, Y., Kaur, A., Suri, B., Singhal, S., 2012. Systematic literature review on regression test prioritization techniques. Informatica 36 (4).
van der Linden, F., Schmid, K., Rommes, E., 2007. Software Product Lines in Action – The Best Industrial Practice in Product Line Engineering. Springer, Berlin, Heidelberg.
Vasic, M., Parvez, Z., Milicevic, A., Gligoric, M., 2017. File-level vs. module-level regression test selection for. NET. In: Proceedings of the 11th Joint Meeting on Foundations of Software Engineering, pp. 848–853.
Wang, S., Ali, S., Gotlieb, A., 2015. Cost-effective test suite minimization in product lines using search techniques. J. Syst. Softw. 103, 370–391.
Wang, S., Ali, S., Gotlieb, A., Liaaen, M., 2016. A systematic test case selection methodology for product lines: results and insights from an industrial case study. Empir. Softw. Eng. 21 (4), 1586–1622.
Wang, S., Ali, S., Gotlieb, A., Liaaen, M., 2017. Automated product line test case selection: industrial case study and controlled experiment. Softw. Syst. Model. 16 (2), 417–441.
Wang, S., Buchmann, D., Ali, S., Gotlieb, A., Pradhan, D., Liaaen, M., 2014. Multi-objective test prioritization in software product line testing: an industrial case study. In: Proceedings of the 18th International Software Product Line Conference-Volume 1, pp. 32–41.
Xu, Z., Cohen, M.B., Motycka, W., Rothermel, G., 2013. Continuous test suite augmentation in software product lines. In: Proceedings of the 17th International Software Product Line Conference, pp. 52–61.
Yoo, S., Harman, M., 2012. Regression testing minimization, selection and prioritization: a survey. Softw. Test. Verif. Reliab. 22 (2), 67–120.
Young, B., Flores, R., Cheatwood, J., Peterson, T., Clements, P., 2017. Product line engineering meets model based engineering in the defense and automotive industries. In: Proceedings of the 21st International Systems and Software Product Line Conference-Volume A, pp. 175–179.
Yu, L., Lei, Y., Nourozborazjany, M., Kacker, R.N., Kuhn, D.R., 2013. An efficient algorithm for constraint handling in combinatorial test generation. Softw. Test. Verif. Valid. (ICST) 242–251.
Yu, L., Ramaswamy, S., 2006. A configuration management model for software product line. INFOCOMP J. Comput. Sci. 5 (4), 1–8.

Pilsu Jung is a student of PhD Program in School of Computing at Korea Advanced Institute of Science and Technology (KAIST). He received his B.S. in Computer Science from Chungnam National University in spring 2008. He received his M.A in Computer Science from KAIST in spring 2012. His research interests are in software testing and software product line engineering.

Sungwon Kang received BA from Seoul National University, Korea in 1982, and received MS and PhD in computer science from the University of Iowa, USA in 1989 and 1992, respectively. From 1993, he was a principal researcher of Korea Telecom R & D Group until October 2001 when he joined Korea Advanced Institute of Science and Technology (KAIST). During 2003–2014, he was an adjunct faculty of Carnegie-Mellon University for the Master of Software Engineering Program. He served as chairs and program chairs of numerous international conferences, the editor of the Korean Journal of Software Engineering Society, and as the president of the Korean Software Engineering Society. His research areas include software architecture, software product line, software testing and data-based software engineering.

Jihyun Lee is an assistant professor of Software Engineering Department, Chonbuk National University, Korea from 2016. She received the BS degree in Information and Communications Engineering from the Chonbuk National University and the MS and the PhD in Computer Sciences from the Chonbuk National University in 2000 and 2005, respectively. She worked as a research assistant professor of School of Computing, KAIST and assistant professor of College of Liberal Arts, Daejeon University. Her research interests include SPL, software testing, software architecture, and business process maturity.